S. P. Mandali's

RAMNARAIN RUIA AUTONOMOUS COLLEGE

MATUNGA, MUMBAI – 400019

DEPARMENT OF COMPUTER SCIENCE & IT

FEILD PROJECT REPORT

Course Code: RPSCS.E515

Meat delivery webapp

Guide Name

Kiran Prajapati

Submitted By

Pius William Ryan

Seat No - 178

MSc Sem II Computer Science 2025-2026

Acknowledgement

- I extend my heartfelt gratitude to my mentor, Kiran Prajapti, whose steady guidance, insightful feedback, and unwavering support were instrumental in shaping and refining the AJ Meat Store WebApp. Their expertise helped me bridge theory with practice and make sound decisions across architecture, data design, security, and user experience.

- I am thankful to my institution and faculty for providing the opportunity, resources, and encouragement to pursue this project. I also appreciate the cooperation of everyone who shared practical inputs on store workflows and customer needs, which informed key features and improved usability.

- Building this application strengthened my skills in the MERN stack, API design, authentication and notifications (including OTP and order confirmations), data modeling in MongoDB, and disciplined deployment practices—learning that will guide my future work.

KP

RUIA COLLEGE
Explore • Experience • Excel

S. P. Mandali's

RAMNARAIN RUIA AUTONOMOUS COLLEGE

Matunga

Department of
Computer Science & Information Technology

Certificate

This is to certify that

Mr. / Miss Pius Ryan *of the*

M.Sc. Computer Science *Class Seat No.* 178 *has*

satisfactorily completed the required number of practicals for

Semester II *in the*

Course Field Project

Course Code RBCGE515 *as per the syllabus*

during the academic year 2025 2026

KP
Subject In-Charge
Date: 16/3

Tawde
Head of Department
Date: 16/3

Examiner
Date:

Ramnarain Ruia Autonomous College Matunga 400019 (1937)
Seal

Index

AJ MEATS **OWNER**

AJ store
Shop No. 3,
Datar Colony,
Ashok Nagar Road,
Bhandup East,
Mumbai-400042, Maharashtra

This is to certify that Mr. Pius Ryan has successfully completed the development of our web application for AJ Meats using the MERN Stack as part of his field project.

The project was delivered as per our requirements and meets our expectations in terms of design, functionality, and performance. We are satisfied with his work and professionalism throughout the development process.

Please note that the online payment gateway is currently in test/beta mode for academic demonstration and evaluation purposes. After the college faculty review, it will be fully activated for live usage.

We appreciate his efforts and wish him success in his future career.

Sincerely,

Warm regards,

+91 90041 61398

Title: Online Meat Store ordering system

A] Problem Definition

Traditional meat shops mostly rely on manual operations such as handwritten records, verbal order-taking, physical billing, and in-shop customer visits. This manual approach leads to challenges such as difficulty in tracking customer orders, lack of proper product categorization, errors in billing, inconsistent pricing, and no digital record of inventory.

The Meat Store Ordering System aims to solve these issues by providing a digital platform where customers can browse meat products, place online orders, make secure payments, and track their order status. The system will also provide an admin dashboard for managing meat categories, stock, orders, and users.

B] Introduction

1. Motivation

Local meat shops face difficulty maintaining manual records of daily orders, stock, pricing, and customer information. Customers often face inconvenience when checking product availability, placing urgent orders, or tracking deliveries. Manual billing increases the chances of errors and slows down the order-handling process.

2. Problem Statement

The current physical and manual workflow of meat shops creates several issues:

• Difficulty tracking customer orders

• No online platform for browsing and buying products

• Inventory shortage often goes unnoticed

• Billing errors due to manual calculations

• No digital records of stock, orders, or users

• Customers cannot track order status

3. Purpose / Objectives and Goals

Objectives:

• To provide an online platform for browsing meat products

• To allow customers to place orders and make online payments

• To maintain a digital customer database

• To manage stock, categories, and product details

• To enable order tracking and automated updates

• To reduce the manual workload of the shop

Goals:

• Reduce human errors in billing and order management

• Improve customer convenience and service speed

• Provide secure online transactions

• Offer a centralized dashboard for admin management

• Build a scalable, modern, and user-friendly system

4. Literature Survey

• Licious – known for product categorization & order tracking

• FreshToHome – offers real-time stock & clean user interface

• Ecommerce store systems – provide cart, checkout & admin models

• Inventory management systems – offer stock management methods

• Authentication systems using JWT – for secure login

5. Project Scope and Limitations

Scope:

• Customer registration, login, and profile management

• Browsing meat products by category

• Add-to-cart, checkout, and secure payment

• Order history

• Admin dashboard for managing products, stock, orders, and users

• Category and product image management

• Online invoices and confirmations

Limitations:

• Designed for a single meat shop, not multi-branch businesses

• Delivery logistics not automated

• No advanced analytics in initial version

• Depends on internet connectivity

• Product quality control is beyond system scope

• real-time order tracking

C.SYSTEM ANALYSIS AND DATA COLLECTION

Online Meat Shop Website

Client Name: AJ Meat Shop

Technology Stack: MERN (MongoDB, Express.js, React.js, Node.js)

Deployment Platform: Netlify (Frontend)

Payment Gateway Status: Pending Integration

1. SYSTEM ANALYSIS

1.1 Introduction

The Online Meat Shop Website for **AJ Meat Mart** is designed to digitize the traditional meat retail business by providing an online platform for customers to browse, select, and order meat products conveniently. The system focuses on hygiene, transparency, and operational efficiency. Currently, online payment gateway integration is pending, and the frontend deployment is planned using Netlify.

1.2 Existing System

The existing system follows a manual process where customers visit local meat shops physically, select products, and make cash payments. There is no digital record of transactions, limited business reach, and no data-driven decision-making.

1.3 Proposed System

The proposed system introduces a web-based solution that enables customers to view categorized meat products with images, manage carts, authenticate securely, and place orders online. This improves customer convenience and business scalability.

1.4 Scope of the System

- Online browsing of meat products
- User authentication (Login/Register)
- Shopping cart functionality
- Order placement (Cash on Delivery)
- Product and inventory management
- Frontend deployment on Netlify
- Online payment gateway (Future Scope)

1.5 Limitations

- Payment gateway not integrated
- Limited delivery coverage

- No real-time order tracking
- Internet dependency

1.6 Stakeholders

- Customers
- Meat Shop Owner
- Delivery Personnel
- Hosting Provider (Netlify)
- Payment Gateway Provider (Future)

2. DATA COLLECTION

2.1 Data Collection Methodology

Primary data was collected using Google Forms to analyze customer preferences, meat consumption patterns, and willingness to order meat online. Responses were automatically stored in Google Sheets.

Survey-based data collection is conducted to identify high-demand meat products, preferred purchasing days, and increased demand during special occasions. The collected data helps analyze customer buying behavior and consumption patterns.

Based on the survey analysis, stock is planned, stored, and made available according to demand to ensure product availability and freshness while reducing wastage. The data is also used to plan targeted discounts and special offers on specific days and occasions, helping achieve wider customer reach, increased sales, and improved profitability for **AJ Meat Mart**.

2.2 Tools Used

- Google Forms
- Google Sheets
- Microsoft Excel
- CSV Files

2.3 Sample Questionnaire

1. Which of these do you buy at least once a month?
2. In a normal month, which single item do you buy the most (by quantity)?
3. On which days do you usually cook meat in your home?
4. In a normal week, how much meat (only chicken/mutton/fish, not eggs) does your household consume in total?

2.4 Data Storage and Processing

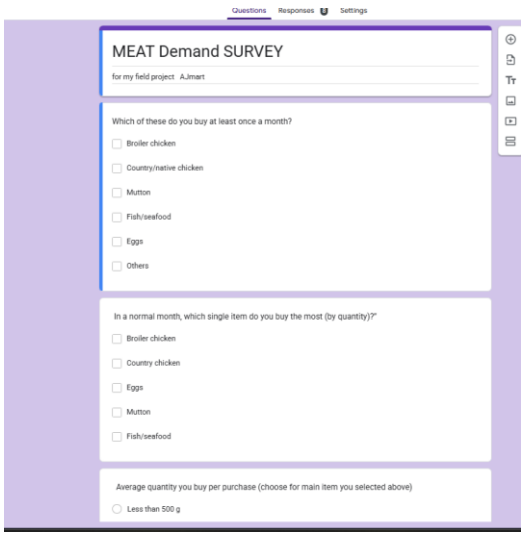
All survey responses were timestamped and stored in Google Sheets. The data was later exported to Excel (.xlsx) and CSV formats for validation and analysis.

2.5 Data Validation

- Duplicate responses removed
- Incomplete entries filtered
- Timestamp verification performed
- Randomized timestamps added for testing

[GOOGLE FORM SCREENSHOT]

Pius RYAN 178



[GOOGLE SHEETS RESPONSE IMAGE]

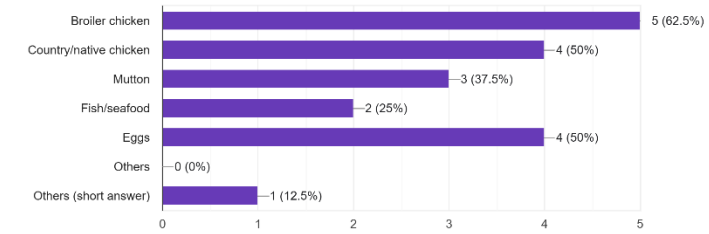
Timestamp	Which of these do you buy at least once a mc	In a normal month, which single item do you	Average quantity you buy per purchase (cho	On which days do you usually cook meat in y	On whi
12/13/2025 18:10:11	Broiler chicken, Eggs	Eggs	500 g – 1 kg	Sunday, Only on festivals/special occasions	No fixed
12/13/2025 18:21:59	Broiler chicken, Mutton, Fish/seafood, Eggs	Eggs	500 g – 1 kg	Thursday,Friday, Sunday	Saturda
12/15/2025 1:38:13	Mutton	Broiler chicken	500 g – 1 kg	Saturday, Sunday	No fixed
12/15/2025 1:56:49	Broiler chicken, Country/native chicken, Mutton, F	Broiler chicken, Country chicken, Eggs, Mutton, Fis	500 g – 1 kg	Monday-Wed, Thursday,Friday, Saturday, Sunday	Saturda
12/15/2025 2:00:34	Country/native chicken, Others (short answer)	Eggs	500 g – 1 kg	Thursday,Friday, Sunday	Monday
12/15/2025 8:58:13	Broiler chicken, Eggs	Country chicken	1–2 kg	Saturday	No fixed
12/15/2025 9:25:44	Country/native chicken	Country chicken	More than 2 kg	Monday-Wed, Thursday,Friday, Saturday, Sunday, (	No fixed
12/15/2025 13:06:49	Broiler chicken, Country/native chicken	Broiler chicken, Country chicken	1–2 kg	Saturday, Sunday	Saturda
12/13/2025 18:10:11	Broiler chicken, Eggs	Eggs	500 g – 1 kg	Sunday, Only on festivals/special occasions	No fixed
12/13/2025 18:21:59	Broiler chicken, Mutton, Fish/seafood, Eggs	Eggs	500 g – 1 kg	Thursday,Friday, Sunday	Saturda
12/15/2025 1:38:13	Mutton	Broiler chicken	500 g – 1 kg	Saturday, Sunday	No fixed
12/15/2025 1:56:49	Broiler chicken, Country/native chicken, Mutton, F	Broiler chicken, Country chicken, Eggs, Mutton, Fis	500 g – 1 kg	Monday-Wed, Thursday,Friday, Saturday, Sunday	Saturda
12/15/2025 2:00:34	Country/native chicken, Others (short answer)	Eggs	500 g – 1 kg	Thursday,Friday, Sunday	Monday
12/15/2025 8:58:13	Broiler chicken, Eggs	Country chicken	1–2 kg	Saturday	No fixed
12/15/2025 9:25:44	Country/native chicken	Country chicken	More than 2 kg	Monday-Wed, Thursday,Friday, Saturday, Sunday, (	No fixed
12/15/2025 13:06:49	Broiler chicken, Country/native chicken	Broiler chicken, Country chicken	1–2 kg	Saturday, Sunday	Saturda
11/27/2025 11:51:40	Broiler chicken, Eggs	Broiler chicken, Country chicken	1–2 kg	Thursday,Friday, Sunday	Monday
12/15/2025 7:02:54	Broiler chicken, Country/native chicken	Eggs	500 g – 1 kg	Saturday	Saturda

[DATA ANALYSIS CHARTS]

Pius RYAN 178

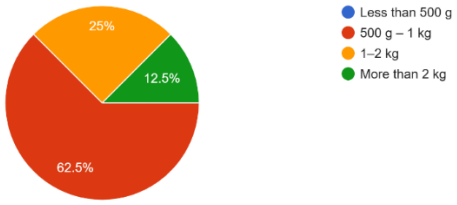
Which of these do you buy at least once a month?

8 responses



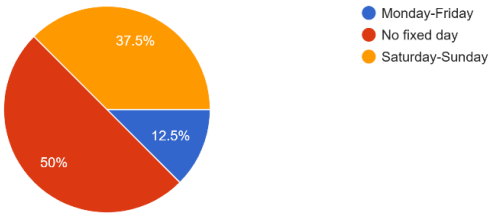
Average quantity you buy per purchase (choose for main item you selected above)

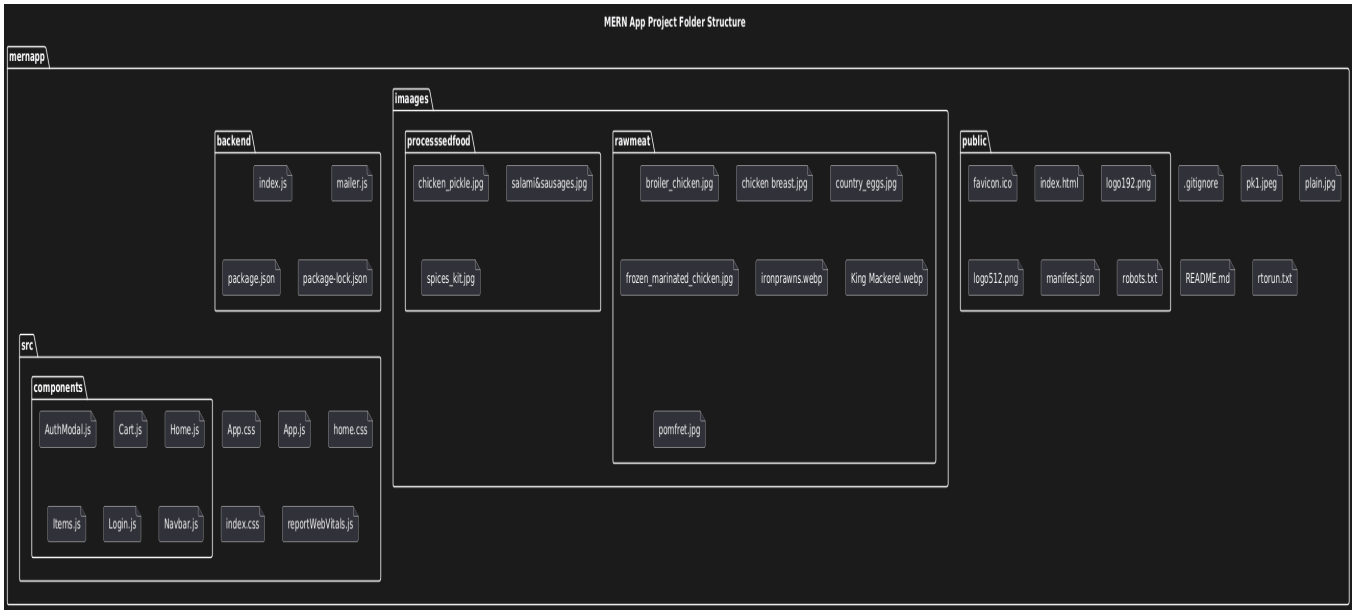
8 responses



On which single day do you buy the maximum quantity in a typical week?

8 responses





---

3. DEPLOYMENT AND FUTURE ENHANCEMENTS

The frontend of the Online Meat Shop Website will be deployed on Netlify, while backend services will be hosted separately. Future enhancements include online payment gateway integration, order tracking, admin dashboards, and mobile application support.

B.System Design (Meat Ordering MERN App)

Overall architecture

Frontend: React single-page app (Create React App) with React Router, components like `Home`, `Items`, `Cart`, `AdminDashboard`, `Track`, `CookieConsent`

Backend: Node.js + Express API in `backend/index.js` using MongoDB (Mongoose) for users, items, orders, and location/cookie data.

Communication: Frontend calls backend over REST (e.g. `/api/auth/me`, `/api/items`, `/api/user/location`) using `fetch`, with `Authorization: Bearer <token>`.

Auth: JWT stored in `localStorage` (`token`), validated on the backend with auth middleware.

Key frontend flows

Customer browses items (`/items`), adds to cart, logs in (`/login`), places order (handled via backend APIs in the main MERN app).

Admin uses `/admin` to manage items, view users and locations, and see order data (admin role from `/api/auth/me`).

`CookieConsent` component tracks cookie/location consent and shows a modal; when accepted it:

Gets location (navigator.geolocation → reverse geocoding).

Calls `/api/user/location` to save `lat`, `lon`, `landmark`, and `cookiesAccepted` ([CookieConsent.js]

Key backend flows

Authentication endpoints (`/api/auth/login`, `/api/auth/me`) issue and verify JWT.

Item/catalog endpoints (`/api/items`, `/api/assets/...`) serve meat items and related assets.

Location and consent endpoint:

- `POST /api/user/location` with JWT:

  - If client sends `lat/lon`, uses them.

  - If not, falls back to IP-based geolocation (`ipapi.co`).

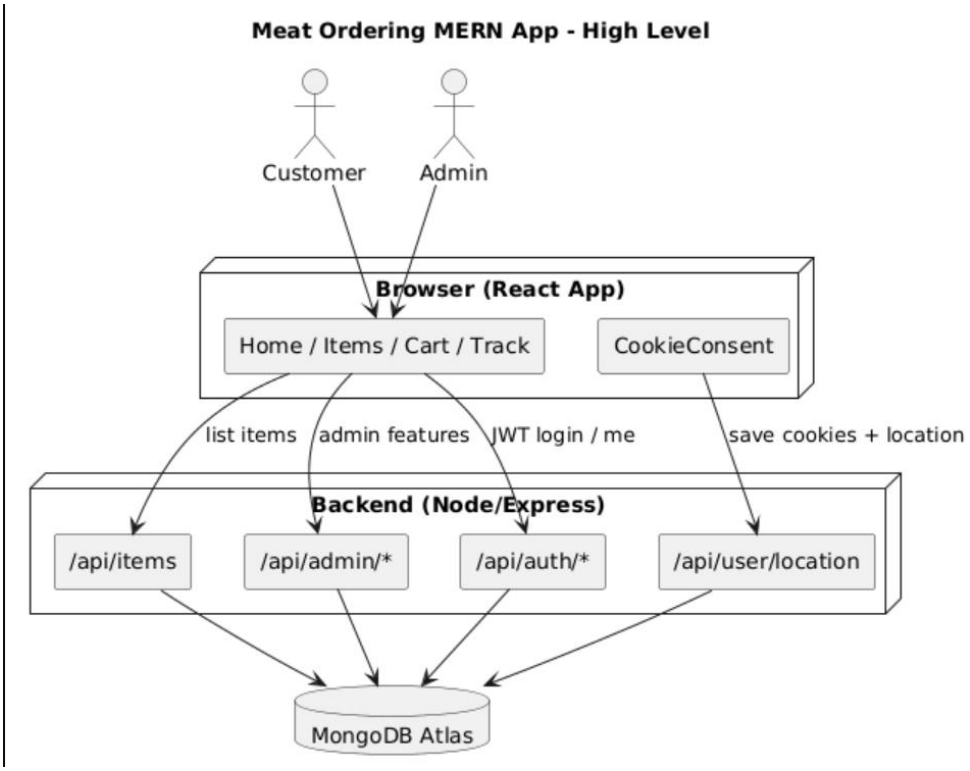
  - Optionally reverse-geocodes to human-readable address and stores `location`, `locationAcc`, `landmark`, `address`, `cookiesAccepted`, `cookiesAcceptedAt` ([index.js](file:///c:/Users/Administrator/Desktop/sem2fielproh/mernapp_cookies/backend/index.js#L935-L1000)).

Admin endpoints for viewing customer locations (`/api/admin/users/locations`) for delivery/coverage decisions.

Data/storage

- MongoDB Atlas (from your `MONGO_URI` in `rtorun.txt`) stores:
  - Users (role: `admin` / `customer`, auth data, cookies/location fields).
  - Items (meat products, prices, etc.).
  - Orders (from the main app).
  - Location fields (`location`, `locationAcc`, `landmark`, `address`, `cookiesAccepted`).

Example high-level diagram



2. Selection of Methodology

Chosen methodology: Agile (Scrum-style, lightweight)

Why Agile fits this project

- Customer-facing e-commerce UI: meat items, offers, and UX change frequently; you need quick iterations on `Home`, `Items`, `Cart`, and admin features.

- MERN stack + simple deploy: React frontend and Node/Express backend are easy to evolve in small increments and redeploy quickly (Vercel/Netlify for frontend, Node backend elsewhere).

- Location/cookies features evolve: consent wording, tracking frequency, and admin maps/reporting will likely change as you learn from real users and delivery behavior.

- Small team: easier to manage 1–2 week sprints with a prioritized backlog (bugs + small features) than a big upfront waterfall spec.

- How to apply it concretely for this project

- Use short sprints (1–2 weeks).

- Maintain one backlog with items like:

  - Improve cookie banner text and flows.

  - Add new meat categories/offers on `Home`.

  - Enhance `Track` page with real-time updates.

  - Better admin view for user locations and orders.

- At the end of each sprint:

  - Deploy updated React build.

  - Deploy backend changes (Express + Mongo).

  - Get feedback from real users/admins and adjust the next sprint’s priorities.

3. Project Context

3.1 Business Problem and Goals

Describe the core business problem:

- What problem does the system solve?

- Who are the primary users?

- What are the main business goals (e.g., increase sales, improve engagement, automate a process)?

3.2 Stakeholders

List key stakeholders:

- End users: Retail customers ordering meat online from AJ Meat Store

- Business owners: AJ Meat Store management and owners

- Operations / Support: AJ Meat Store operations team (order handling, packing, delivery coordination) and customer support staff

- External systems / partners: Payment gateway provider, SMS/email notification provider, and delivery partners/courier services

3.3 High-Level Scope

In Scope:

- Feature A: Online meat catalog with search, categories (chicken, mutton, etc.), and product details

- Feature B: Cart and checkout flow with address management, delivery slot selection, and order placement

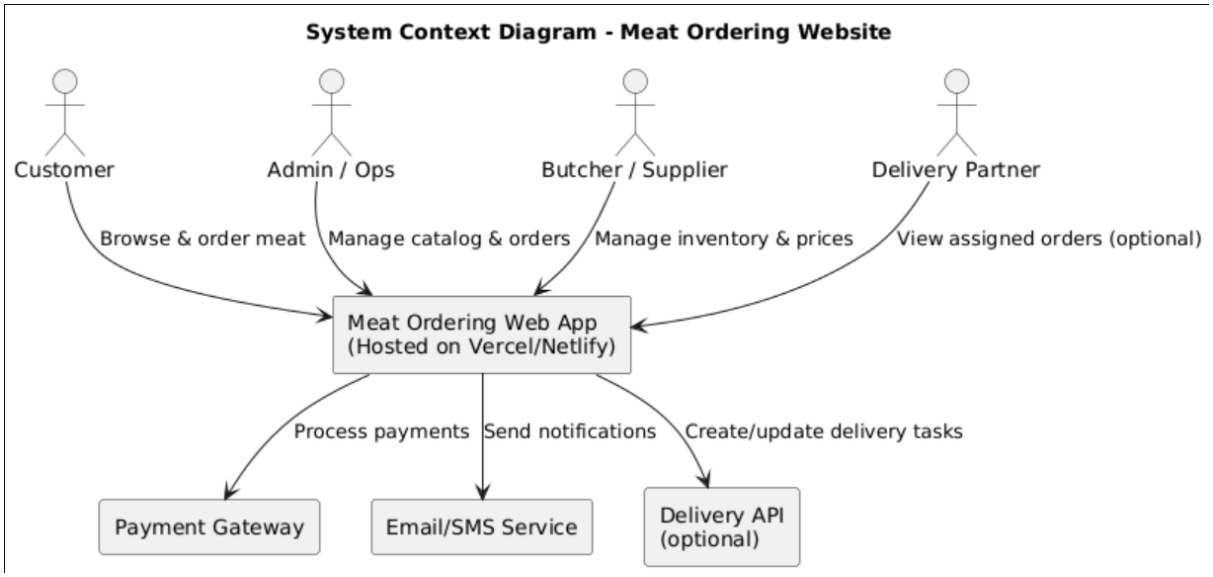
- Feature C: Basic admin panel to manage items (add/edit/delete), prices, and view customer orders

Out of Scope:

- Advanced features like loyalty points, subscriptions, and referral programs

- Full delivery logistics/route optimization and integration with multiple external courier providers

3.4 System Context Diagram



4. Functional Requirements (Key Features)

- Users can browse the meat catalog (by category), add items to a cart, register/login, place orders, and view basic order status.

- Admins can manage products (add/edit/remove items, prices, availability), view customer orders, and update order status (e.g., pending, preparing, delivered).

- The system must securely handle user authentication, calculate accurate order totals, process payments via a payment gateway, and store customer location/consent for delivery planning.

4.1 Non-Functional Requirements

- Performance: Page load time under **3 seconds** for typical users.

- Scalability: Able to handle at least **50–100 concurrent users** (sufficient for demo and early real usage).

- Availability: Target uptime of **≈99%** during normal operation (within the limits of free hosting tiers).

- Security: Use HTTPS, secure authentication (hashed passwords/JWT), and proper authorization for admin vs customer.

- Compliance: Basic data protection (do not expose passwords or payment data; rely on the payment gateway for card handling) and follow local rules for online sale of meat.

4.2 Constraints

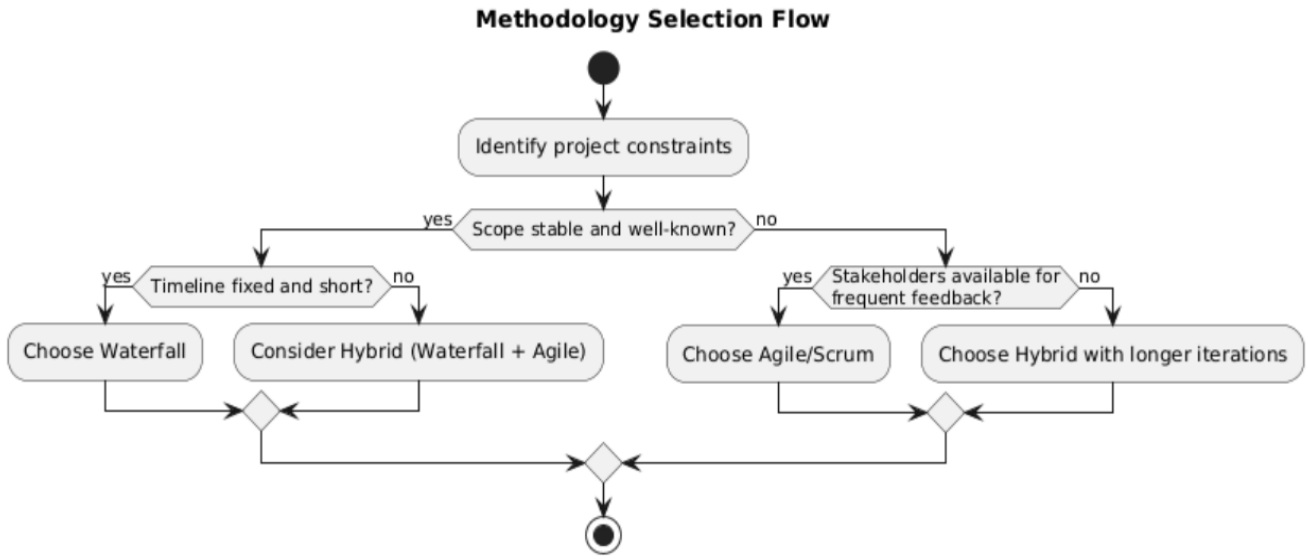
- Time: Must be delivered by the **college project submission date / end of this semester**.

- Budget: **Zero budget** – must rely on free or student tiers of all services (Vercel/Netlify, MongoDB Atlas, email/SMS where possible).

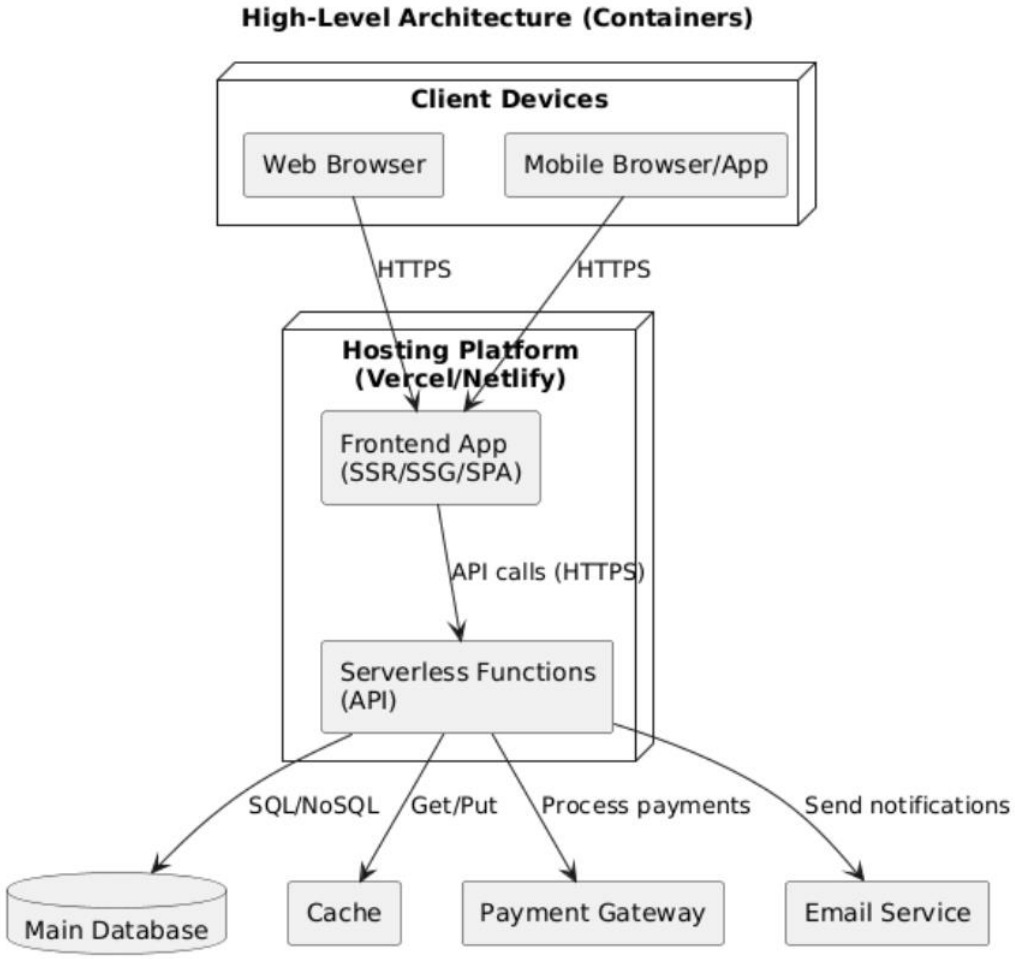
- Technology: Web app hosted on **Vercel or Netlify**, using **React (frontend) + Node.js/Express (backend) + MongoDB Atlas (database)**.

- Team: **Solo builder** (single developer – you), doing full-stack work as part of a college field project

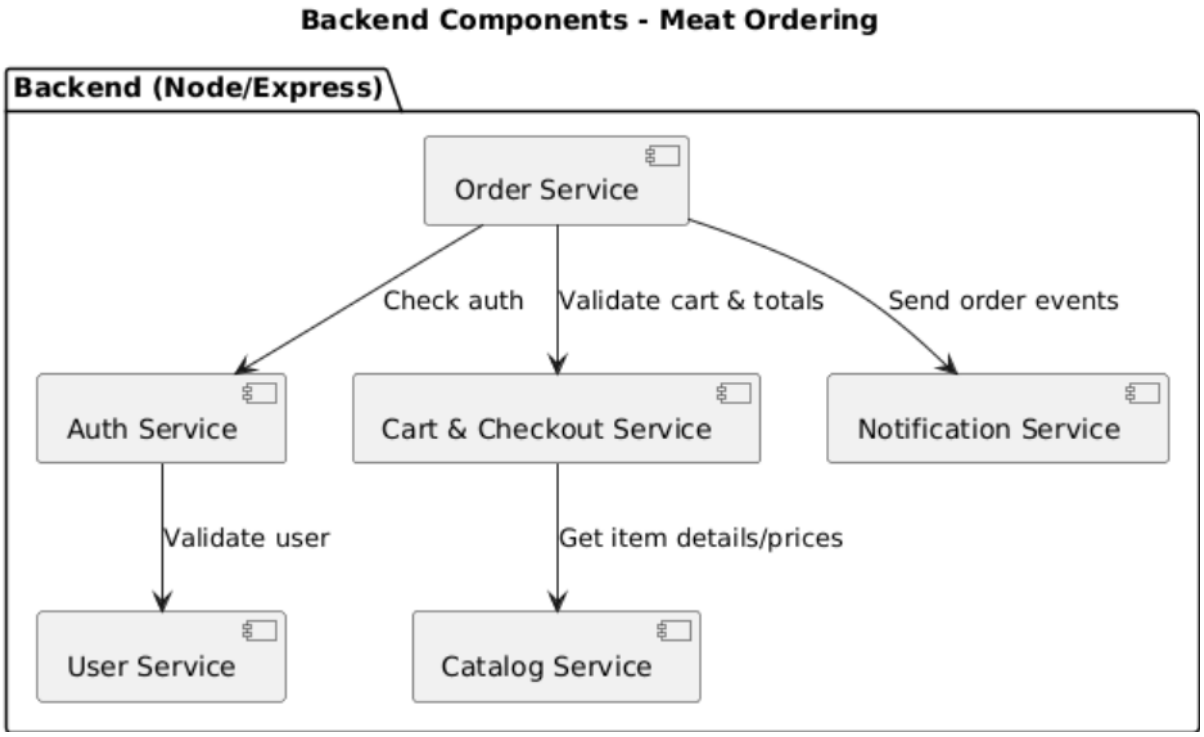
5.Methodology Selection Flow



5.1 Container-Level Diagram

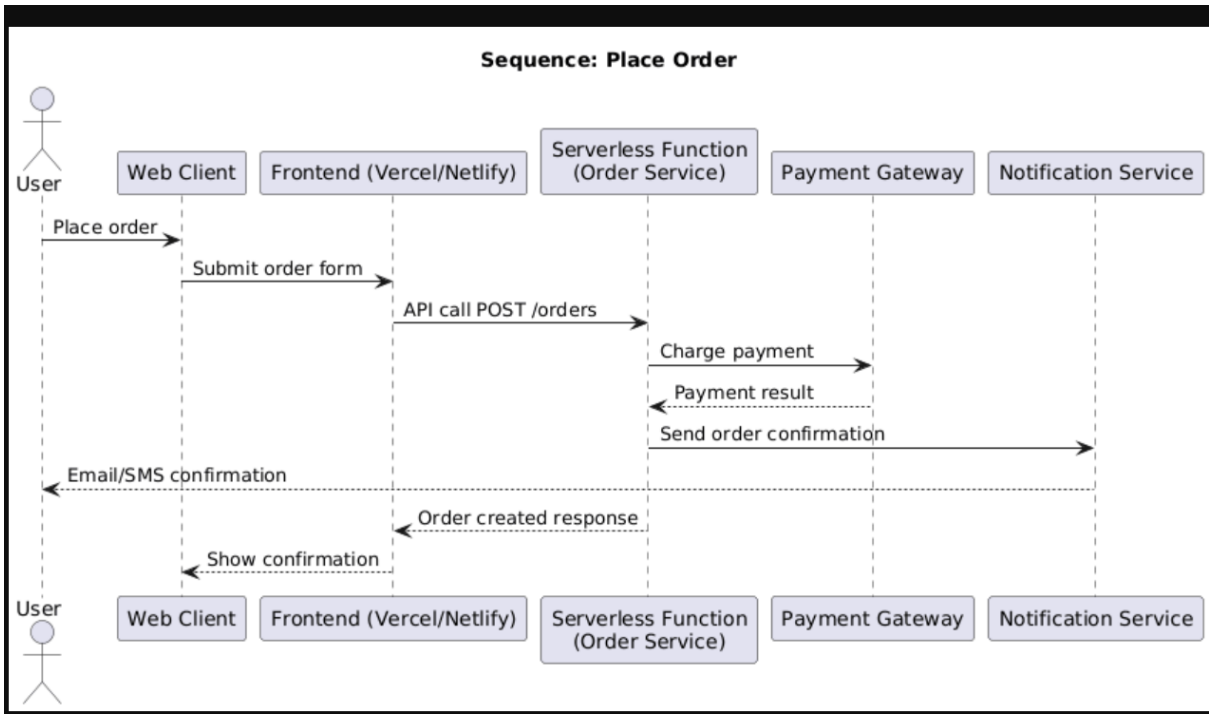


5.2 Component Diagram



5.3 Key Use Case: Example Sequence Diagram

Example use case: User places an order



6.Data design

6.1 Main Entities

Core entities in the meat ordering system (current scope):

User

- Stores login / identity and role (customer or admin).
- Key fields: `id`, `email/username`, `passwordHash`, `role`, `createdAt`.

Item

- A meat product the customer can order.
- Key fields: `id`, `name`, `category`, `cutType`, `pricePerUnit`, `available`, `createdAt`.

Cart

- Temporary basket of items before placing an order (per user).
- Key fields: `id`, `userId`, list of `{ itemId, quantity }`, `updatedAt`.

Order

- A confirmed purchase made by a user.
- Key fields: `id`, `userId`, delivery details, `totalAmount`, `status`, `createdAt`.

OrderItem

- Line items inside an order.
- Key fields: `id`, `orderId`, `itemId`, `quantity`, `unitPrice`, `subtotal`.

OrderStatusHistory

- History of status changes for an order.
- Key fields: `id`, `orderId`, `status`, `changedAt`, `changedBy`.

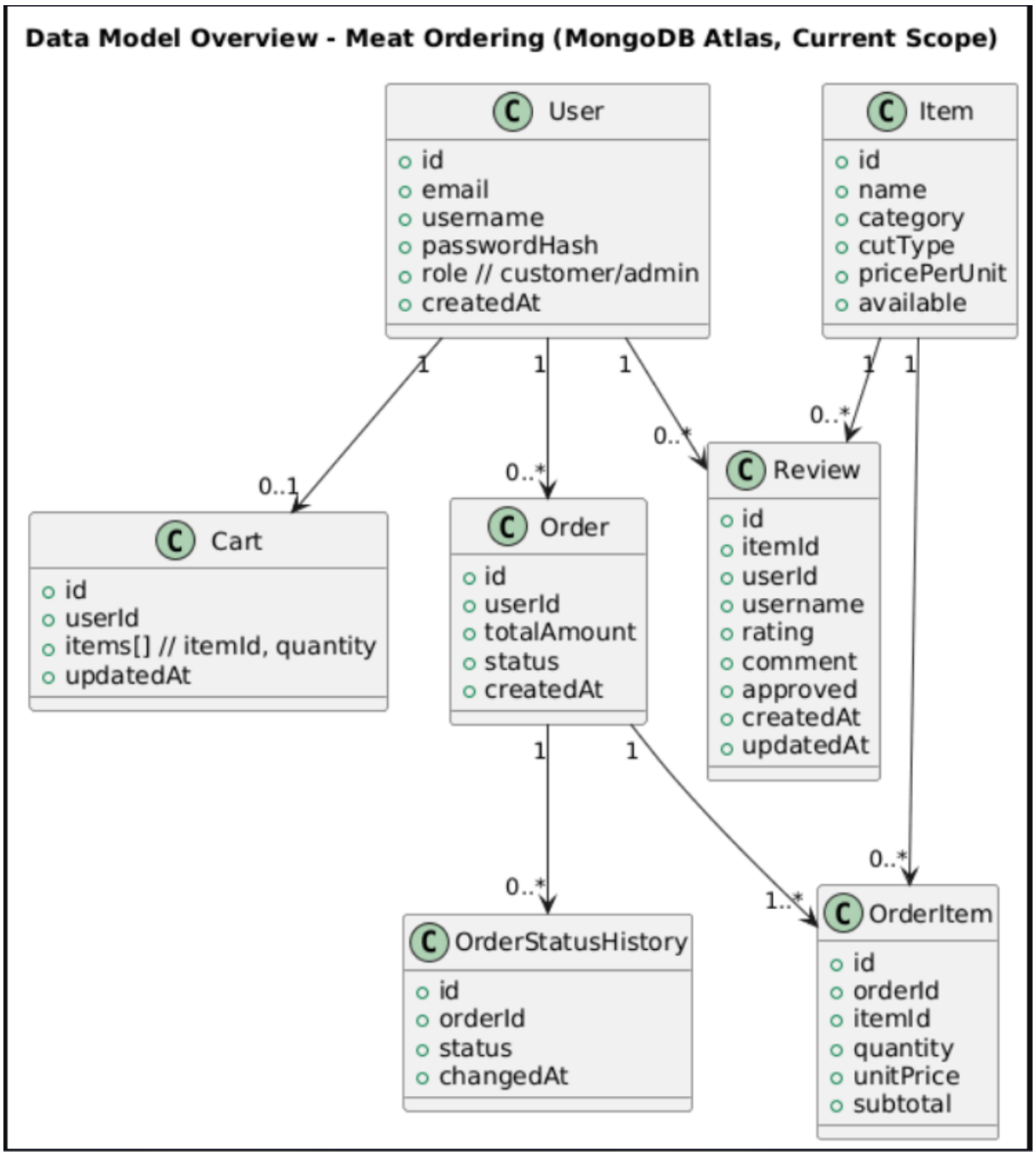
Review

- Rating and comment a user leaves for an item.

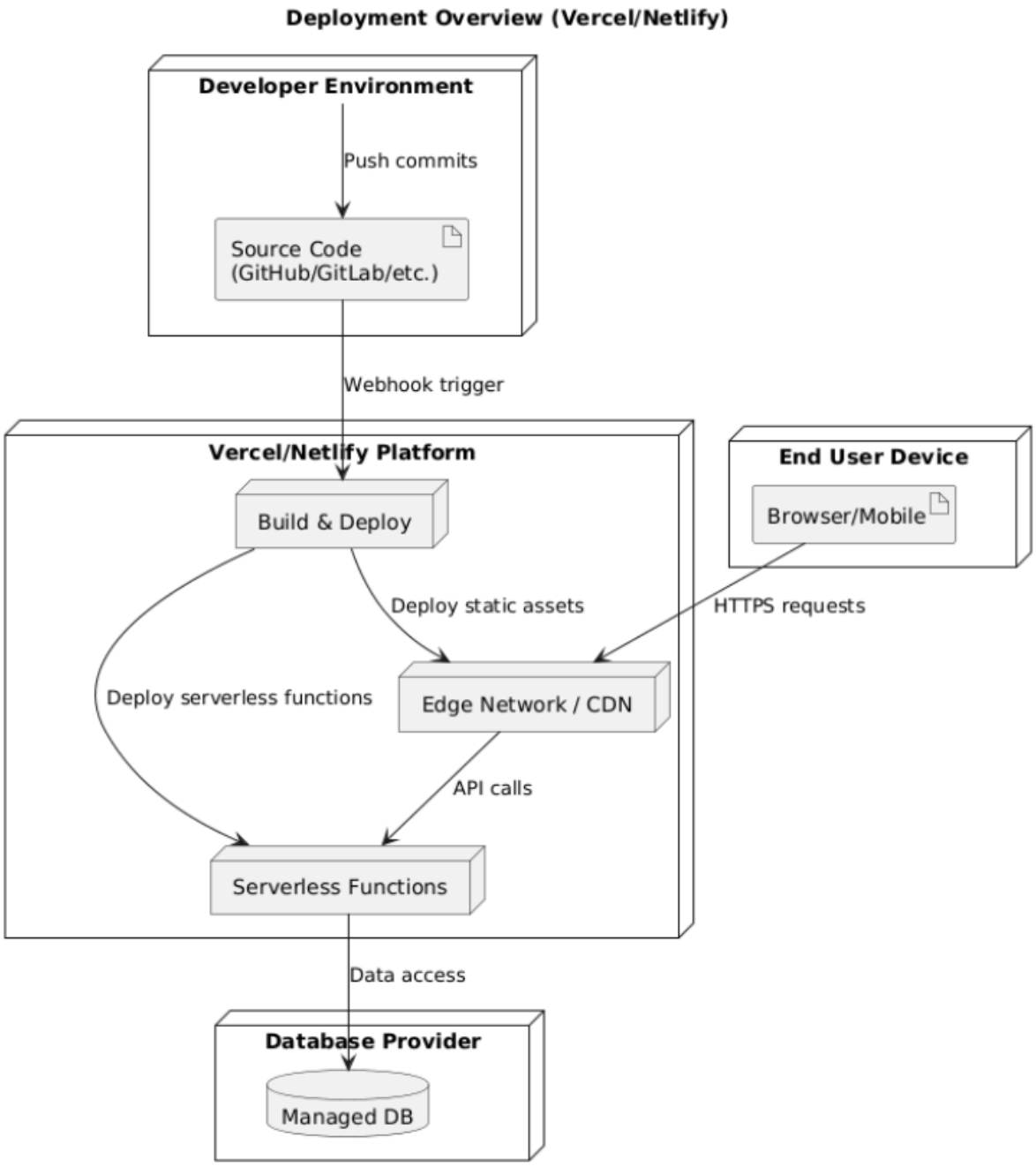
- Key fields: `id`, `itemId`, `userId`, `username`, `rating`, `comment`, `approved`, `createdAt`, `updatedAt`.

> Coupons are treated as **future enhancement**, not part of the current data model scope.

6.2 Data Model Diagram

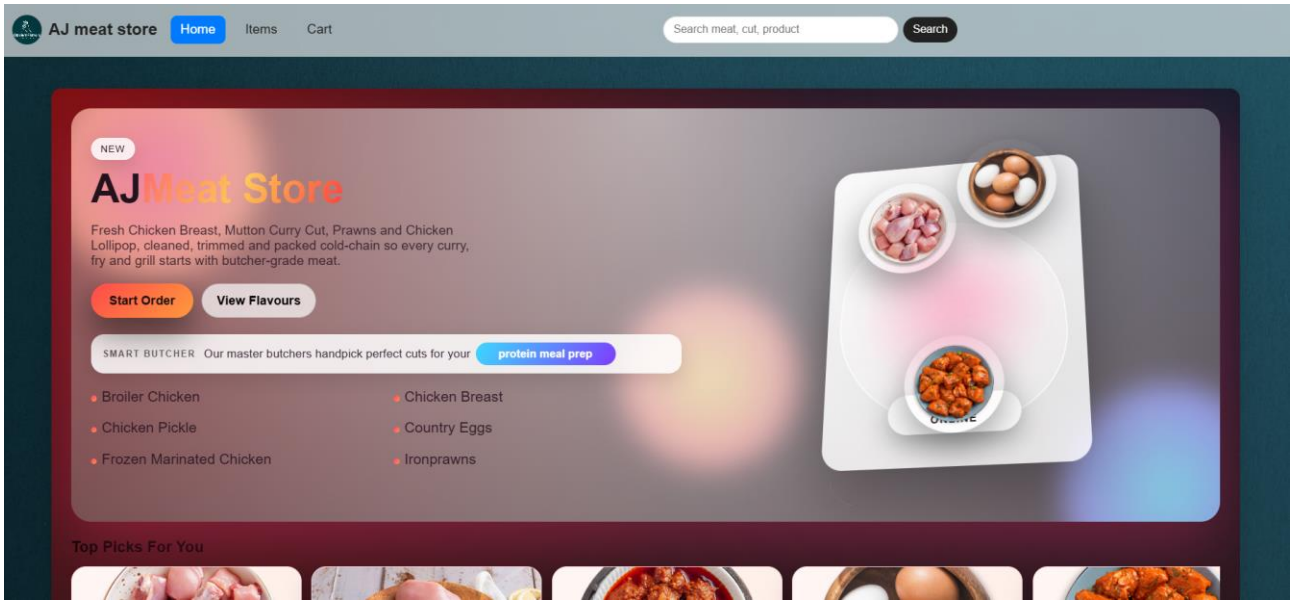


- Future Scope: Implement coupon and discount system (promo codes applied at checkout).

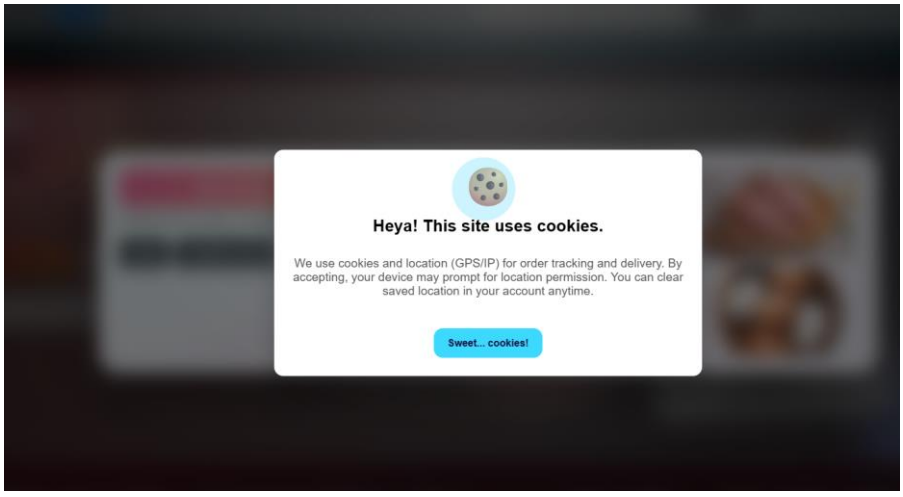

Deployment Overview (Vercel/Netlify)
Developer Environment
Push commits
Source Code
(GitHub/GitLab/etc.)
Webhook trigger
Vercel/Netlify Platform
Build & Deploy
End User Device
Browser/Mobile
Deploy static assets
HTTPS requests
Deploy serverless functions
Edge Network / CDN
API calls
Serverless Functions
Data access
Database Provider
Managed DB

7. User Interfaces

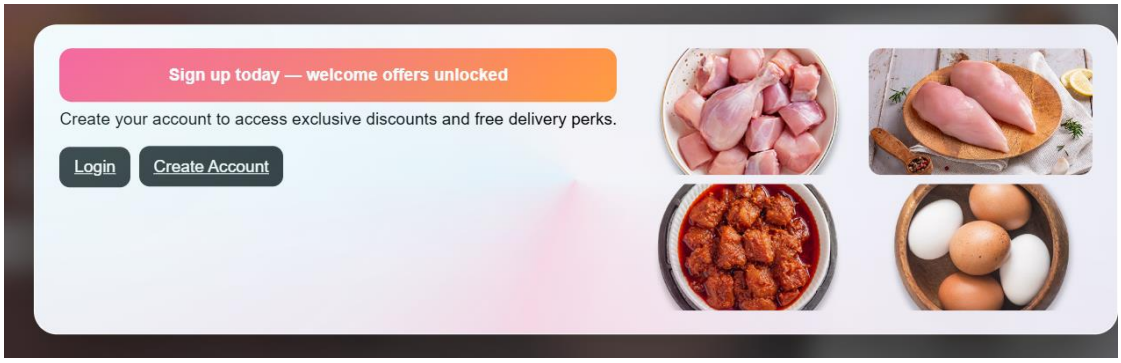
7.1 Home page



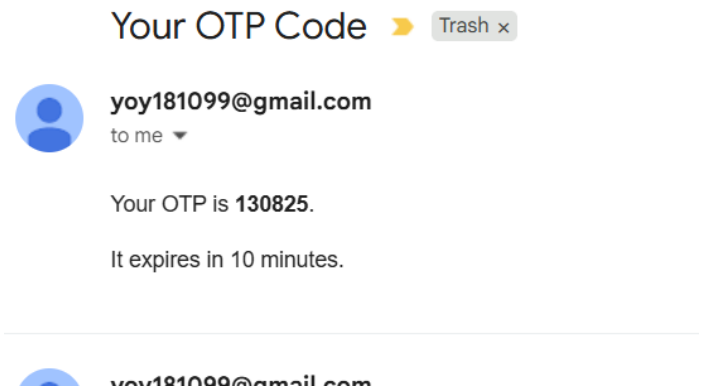
7.2 Cookies notification

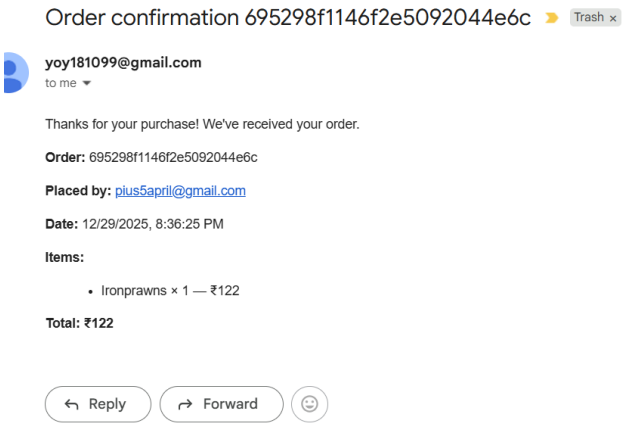


7.3 Auth pop-up

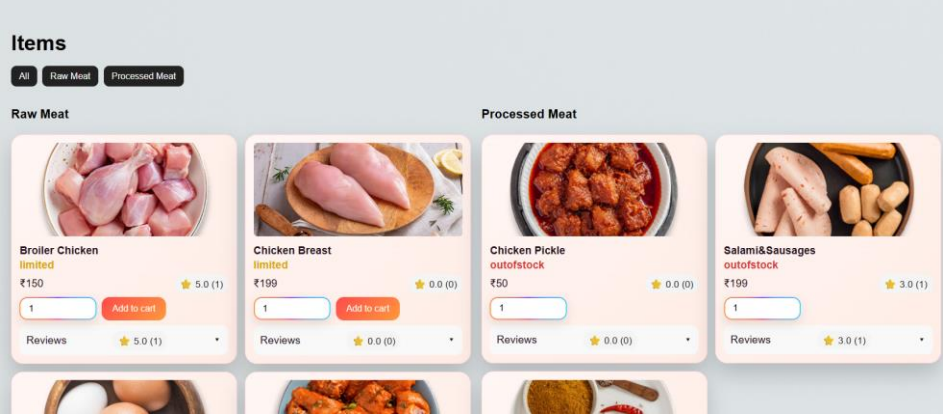


7.4 SMTP server mail(OTP & Order confirmation)

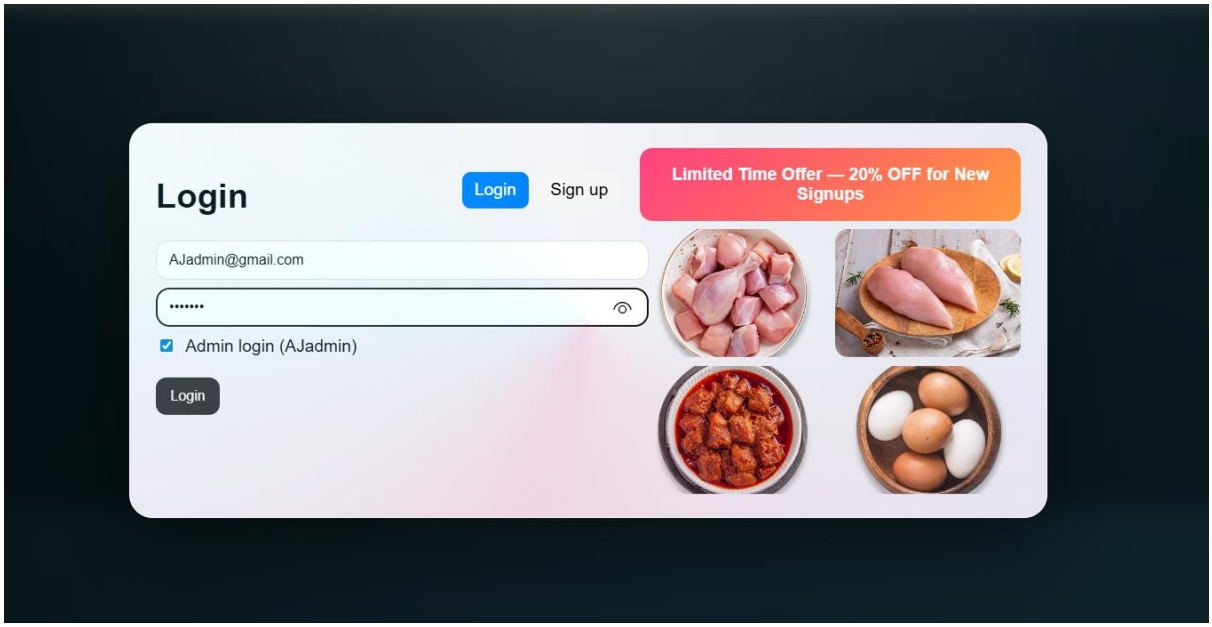




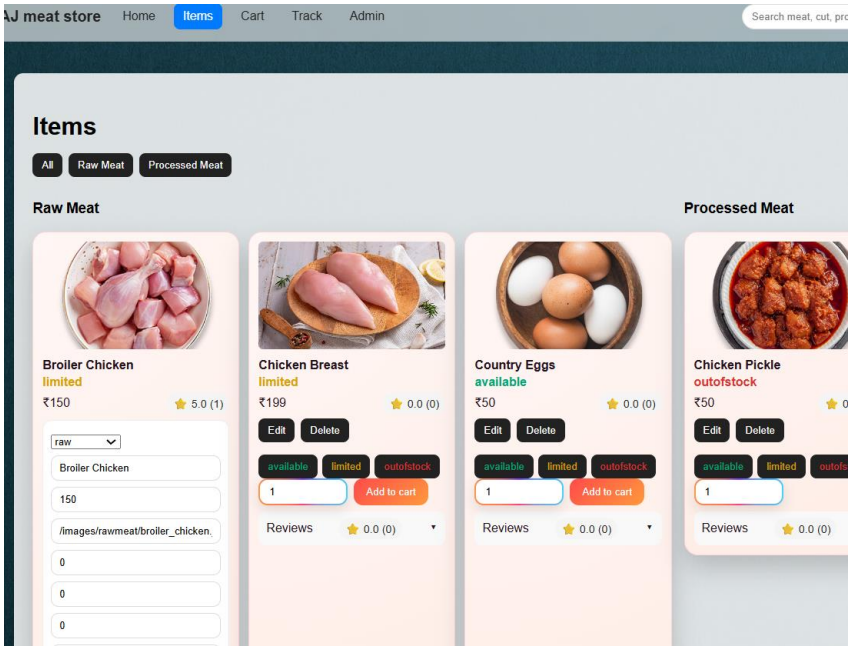
7.5 Cart page

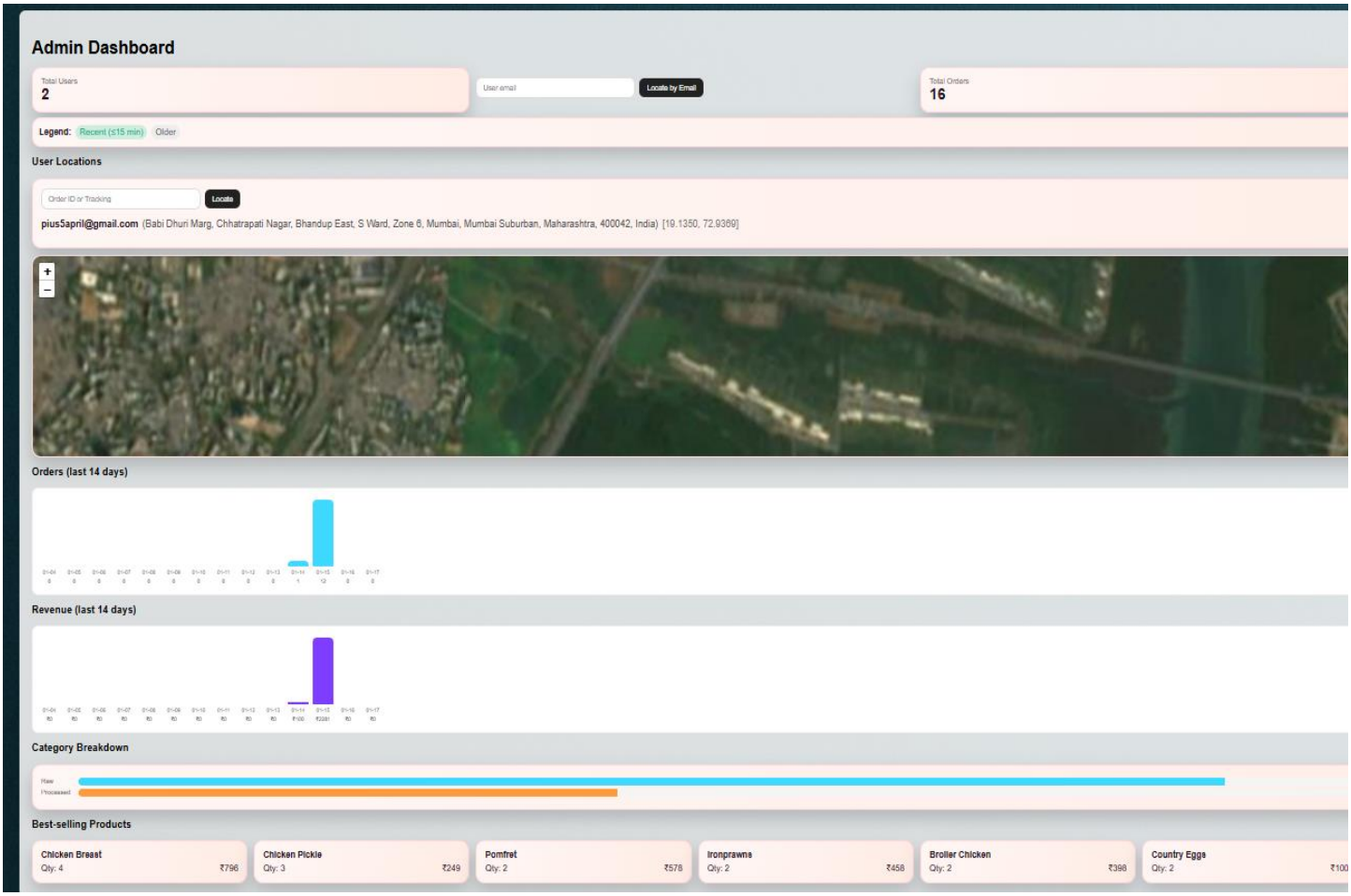


7.6 Admin pages



Pius RYAN 178





8.Appendix

Layer	Technology / Tool	Details / Purpose
Frontend	React (Create React App)	Builds the single-page UI for meat ordering (Home, Items, Cart, Admin, Track).
Frontend	JavaScript (ES6+)	Main programming language for frontend logic and components.
Frontend	react-router-dom	Handles client-side routing between pages (/, /items, /cart, /admin, /track).
Backend	Node.js	JavaScript runtime for executing server-side code.
Backend	Express.js	Web framework for building REST APIs (auth, items, orders, location, reviews).
Backend	JWT (JSON Web Token)	Used for authentication and authorization between frontend and backend.
Database	MongoDB Atlas	Cloud-hosted NoSQL database storing users, items, carts, orders, status histories, and reviews.
Database	Mongoose	ODM library to define schemas and interact with MongoDB from Node.js.
Deployment / Hosting	Vercel / Netlify (planned)	Hosts the built React frontend as a static single-page application.
Deployment / Hosting	Node/Express Server	Runs the backend API with environment variables (e.g., MONGO_URI, JWT_SECRET).
Tools	npm	Package manager for installing dependencies and running scripts.
Tools	react-scripts	Build and development tooling for the React app (start, build, test).
Tools	Git	Version control system for managing source code.

D. Implementation Details

IMPORTANT NOTE

Disclaimer: This is a client-based project developed for a specific business use case ("AJ Meat Store"). As such, certain proprietary configurations, sensitive environment variables (such as API keys, database credentials, and payment secrets), and specific business logic implementations have been omitted or redacted from this documentation to protect client confidentiality and system security. The source code provided above serves as an architectural reference and does not represent the complete, deployable codebase.

1. Software Specifications

This project is a full-stack MERN (MongoDB, Express, React, Node.js) application designed to provide a scalable, secure, and responsive web-based system.

Backend Technologies

- Node.js – Server-side runtime environment
- Express.js – REST API framework
- MongoDB – NoSQL database
- Mongoose – ODM for MongoDB
- JWT & Bcrypt – Authentication and security
- Stripe – Payment gateway integration
- Nodemailer / Resend – Email services

Frontend Technologies

- React.js – User interface development
- React Router – Client-side routing
- CSS3 & Framer Motion – Styling and animations
- MapLibre GL – Geolocation and maps

2. Hardware Specifications

1. Server Requirements:
    - Minimum 1 vCPU
    - 1 GB RAM recommended
    - 500 MB SSD Storage
    - Stable internet connection with SSL support
2. Client Requirements:
    - Desktop / Laptop / Mobile device
    - Any modern web browser
    - Internet connection

3. Source Code

The project follows a modular MERN architecture with separate backend and frontend directories. The backend handles APIs, authentication, and database operations, while the frontend manages UI rendering and client-side logic.

- Key folders include backend/, src/, public/, and configuration files.

C:\Users\Administrator\Documents\GitHub\mernapp\backend
Index.js

```
1   const express = require('express');
2   const mongoose = require('mongoose');
3   const cors = require('cors');
4   const morgan = require('morgan');
5   const bcrypt = require('bcryptjs');
6   const jwt = require('jsonwebtoken');
7   const fs = require('fs');
8   const path = require('path');
9   const { sendOtpEmail, sendOrderEmail } = require('./mail
10  const Stripe = require('stripe');
11  const multer = require('multer');
12  const cloudinary = require('cloudinary').v2;
13
14  const app = express();
15  app.use(morgan('dev'));
16  const port = process.env.PORT || 5000;
17  const JWT_SECRET = process.env.JWT_SECRET || 'dev-secret
18
19  const FRONTEND_ORIGIN = process.env.FRONTEND_ORIGIN || '
20  app.use(cors({
21    origin: (origin, callback) => {
22      const allowed = FRONTEND_ORIGIN.split(',').map(o =>
23      // Allow exact matches or Vercel preview URLs for th
24      const isAllowed = !origin || allowed.includes(origin
```

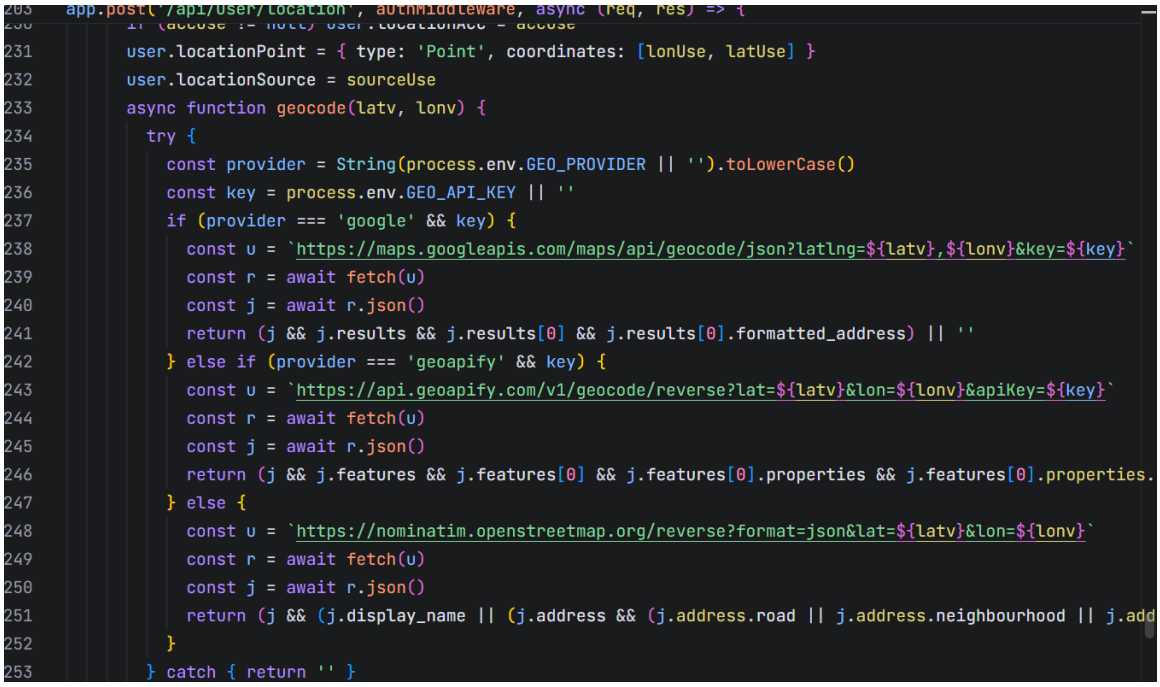
```
  },
  credentials: false
}));
app.use(express.json());

// Helper to resolve asset paths (checks current dir first, then parent)
const getAssetPath = (...segments) => {
  const localPath = path.join(__dirname, ...segments);
  if (fs.existsSync(localPath)) return localPath;
  return path.join(__dirname, '..', ...segments);
};

app.use('/images', express.static(getAssetPath('imaages')));
app.use('/offerimages', express.static(getAssetPath('offerimages')));
app.get('/site-assets/plain.jpg', (req, res) => {
  res.sendFile(getAssetPath('plain.jpg'))
});
app.get('/site-assets/AJ.png', (req, res) => {
  res.sendFile(getAssetPath('AJ.png'))
});

const MONGO_URI = process.env.MONGO_URI;
if (!MONGO_URI) {
  console.error('MONGO_URI environment variable is not set');
  process.exit(1);
```

```
const userSchema = new mongoose.Schema(
  {
    username: { type: String, required: true, unique: true, trim: true },
    email: { type: String, unique: true, sparse: true },
    role: { type: String, enum: ['admin', 'customer'], default: 'customer' },
    emailVerified: { type: Boolean, default: false },
    passwordHash: { type: String },
    otpCode: { type: String },
    otpExpires: { type: Date },
    cookiesAccepted: { type: Boolean, default: false },
    cookiesAcceptedAt: { type: Date },
    location: {
      lat: { type: Number },
      lon: { type: Number },
    },
    locationAcc: { type: Number },
    landmark: { type: String, default: '' },
    locationPoint: {
      type: { type: String, enum: ['Point'], default: 'Point' },
      coordinates: { type: [Number] }
    },
    locationSource: { type: String, enum: ['GPS', 'IP'], default: 'IP' },
    address: { type: String, default: '' },
```

```
);

const cartSchema = new mongoose.Schema(
  {
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, unique: true },
    items: { type: [cartItemSchema], default: [] },
  },
  { timestamps: true }
);

const orderSchema = new mongoose.Schema(
  {
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    username: { type: String, required: true },
    items: { type: [cartItemSchema], default: [] },
    totalAmount: { type: Number, required: true },
    discountAmount: { type: Number, default: 0 },
    appliedCoupon: { type: String, default: '' },
    status: { type: String, enum: ['confirmed', 'shipped', 'delivered', 'cancelled'], default: 'confirmed' },
    trackingNumber: { type: String, unique: true, sparse: true },
  },
  { timestamps: true }
);
```

```
}

async function importItemsFromImages() {
  const base = path.join(__dirname, '..', 'imaages');
  const rawDir = path.join(base, 'rawmeat');
  const processedDir = path.join(base, 'processsedfood');

  const entries = [];
  function priceFor(category, file) {
    const f = file.toLowerCase();
    if (category === 'raw') {
      if (f.includes('egg')) return 99;
      if (f.includes('prawn')) return 229;
      if (f.includes('mackerel')) return 279;
      if (f.includes('pomfret')) return 289;
      if (f.includes('broiler')) return 199;
      if (f.includes('breast')) return 199;
      if (f.includes('frozen') || f.includes('marinated')) return 249;
      return 299;
    } else {
      if (f.includes('chicken_salami')) return 199;
      if (f.includes('pickle')) return 149;
      if (f.includes('spice')) return 149;
      return 199;
    }
```

```
1200  })
1201
1202  // User location & cookies consent
1203  app.post('/api/user/location', authMiddleware, async (req, res) => {
1204    try {
1205      const { lat, lon, accuracy, landmark, cookiesAccepted, source } = req.body || {};
1206      const user = await User.findById(req.userId);
1207      if (!user) return res.status(404).json({ error: 'User not found' });
1208      let latUse = lat != null ? Number(lat) : null
1209      let lonUse = lon != null ? Number(lon) : null
1210      let accUse = accuracy != null ? Math.max(0, Number(accuracy)) : null
1211      let sourceUse = source || 'GPS'
1212
1213      if (latUse == null || lonUse == null) {
1214        try {
1215          let ip = ''
1216          const xf = (req.headers['x-forwarded-for'] || req.headers['x-real-ip'] || '').toString()
1217          if (xf) ip = xf.split(',')[0].trim()
1218          if (!ip && req.socket && req.socket.remoteAddress) ip = String(req.socket.remoteAddress).replace('::
1219          const url = ip ? `https://ipapi.co/${encodeURIComponent(ip)}/json` : 'https://ipapi.co/json'
1220          const r = await fetch(url)
1221          const j = await r.json()
1222          latUse = j && j.latitude != null ? Number(j.latitude) : latUse
1223          lonUse = j && j.longitude != null ? Number(j.longitude) : lonUse
```

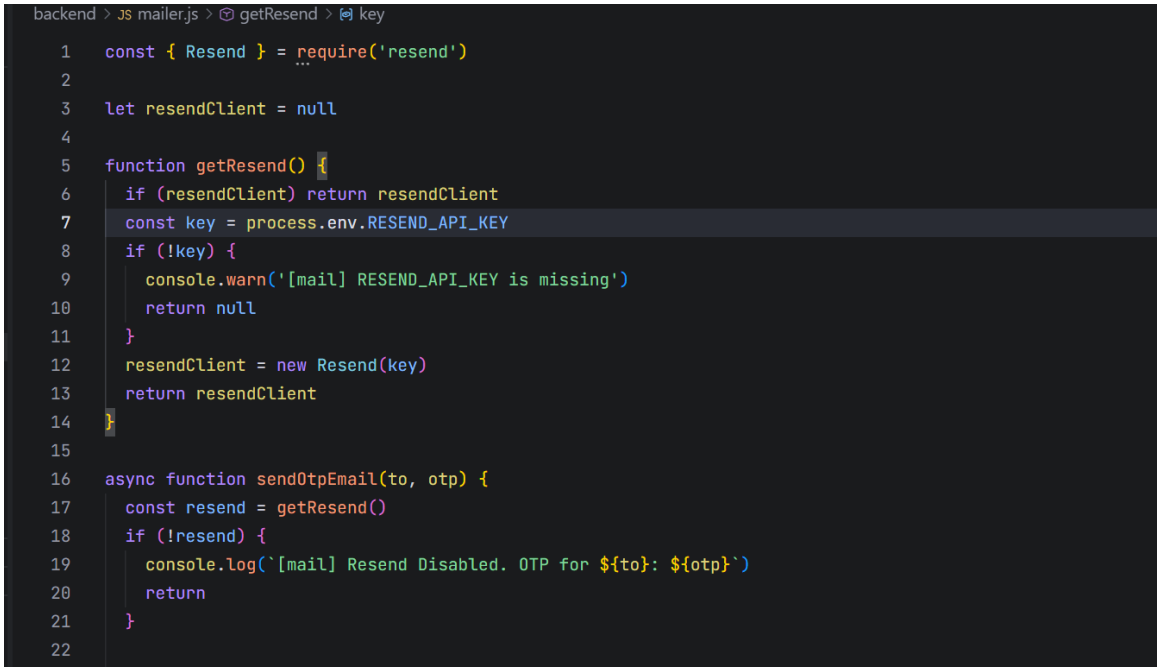


```
203   app.post('/api/user/location', authMiddleware, async (req, res) => {
231       user.locationPoint = { type: 'Point', coordinates: [lonUse, latUse] }
232       user.locationSource = sourceUse
233       async function geocode(latv, lonv) {
234         try {
235           const provider = String(process.env.GEO_PROVIDER || '').toLowerCase()
236           const key = process.env.GEO_API_KEY || ''
237           if (provider === 'google' && key) {
238             const u = `https://maps.googleapis.com/maps/api/geocode/json?latlng=${latv},${lonv}&key=${key}`
239             const r = await fetch(u)
240             const j = await r.json()
241             return (j && j.results && j.results[0] && j.results[0].formatted_address) || ''
242           } else if (provider === 'geoapify' && key) {
243             const u = `https://api.geoapify.com/v1/geocode/reverse?lat=${latv}&lon=${lonv}&apiKey=${key}`
244             const r = await fetch(u)
245             const j = await r.json()
246             return (j && j.features && j.features[0] && j.features[0].properties && j.features[0].properties.
247           } else {
248             const u = `https://nominatim.openstreetmap.org/reverse?format=json&lat=${latv}&lon=${lonv}`
249             const r = await fetch(u)
250             const j = await r.json()
251             return (j && (j.display_name || (j.address && (j.address.road || j.address.neighbourhood || j.add
252           }
253         } catch { return '' }
```

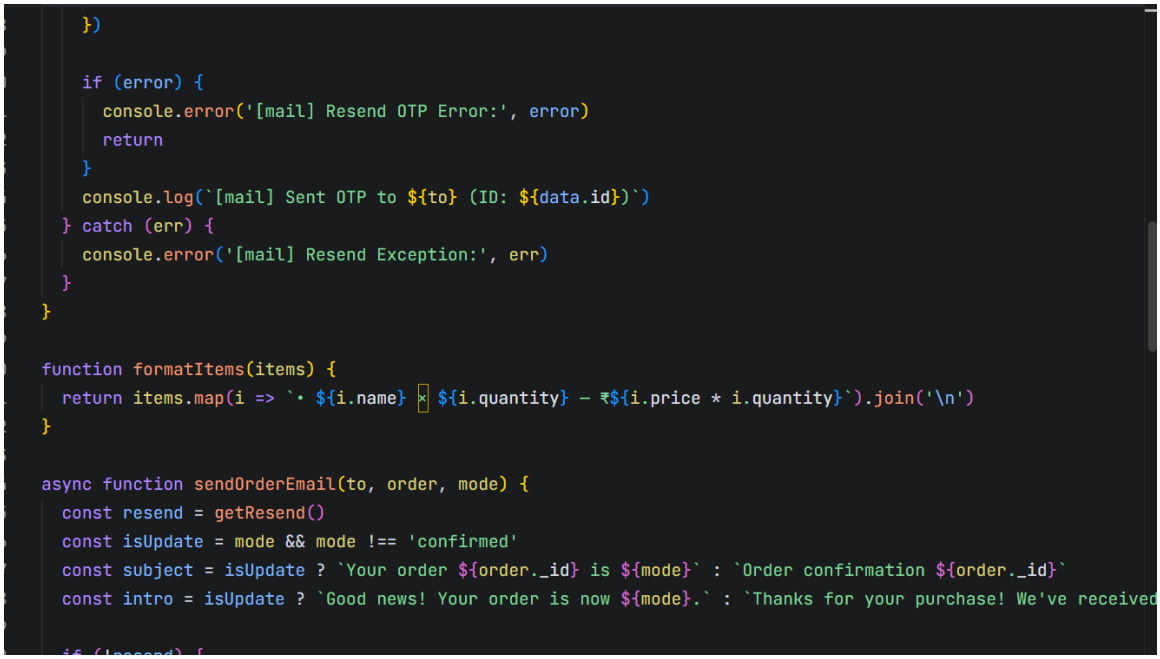
```
1270  app.get('/api/admin/users/locations', authMiddleware, async (req, res) => {
1280      // Get active carts to link with users
1281      const activeCarts = await Cart.find({ 'items.0': { $exists: true } }).select('userId items');
1282      const cartMap = new Map();
1283      activeCarts.forEach(c => {
1284        const total = c.items.reduce((sum, i) => sum + (i.price * i.quantity), 0);
1285        cartMap.set(String(c.userId), { count: c.items.length, total });
1286      });
1287
1288      res.json(users.map(u => {
1289        const cart = cartMap.get(String(u._id));
1290        return {
1291          _id: u._id,
1292          username: u.username,
1293          lat: u.location && u.location.lat != null ? u.location.lat : null,
1294          lon: u.location && u.location.lon != null ? u.location.lon : null,
1295          acc: u.locationAcc != null ? u.locationAcc : null,
1296          src: u.locationSource || 'IP',
1297          landmark: u.landmark || '',
1298          address: u.address || '',
1299          updatedAt: u.updatedAt,
1300          cartCount: cart ? cart.count : 0,
1301          cartTotal: cart ? cart.total : 0
1302        };
1303      }));
```

```
270  app.get('/api/admin/users/locations', authMiddleware, async (req, res) => {
306      res.status(500).json({ error: 'Fetch user locations failed' })
307    }
308  });
309  app.get('/api/admin/user/location', authMiddleware, async (req, res) => {
310    try {
311      const me = await User.findById(req.userId);
312      if (!me || me.role !== 'admin' || me.username !== 'AJadmin') return res.status(403).json({ error: 'Admin
313      const email = String((req.query && req.query.email) || '').trim().toLowerCase();
314      if (!email) return res.status(400).json({ error: 'Email required' });
315      const user = await User.findOne({ email });
316      if (!user) return res.status(404).json({ error: 'User not found' });
317      const loc = user.location || {};
318      res.json({
319        username: user.username,
320        email: user.email || '',
321        userLocation: { lat: loc.lat ?? null, lon: loc.lon ?? null, acc: user.locationAcc ?? null },
322        userLandmark: user.landmark || ''
323      });
324    } catch (e) {
325      res.status(500).json({ error: 'Lookup failed' })
326    }
327  });
328
```

Mailer.js



```
backend > JS mailer.js > getResend > key
const { Resend } = require('resend')

let resendClient = null

function getResend() {
  if (resendClient) return resendClient
  const key = process.env.RESEND_API_KEY
  if (!key) {
    console.warn('[mail] RESEND_API_KEY is missing')
    return null
  }
  resendClient = new Resend(key)
  return resendClient
}

async function sendOtpEmail(to, otp) {
  const resend = getResend()
  if (!resend) {
    console.log(`[mail] Resend Disabled. OTP for ${to}: ${otp}`)
    return
  }
```



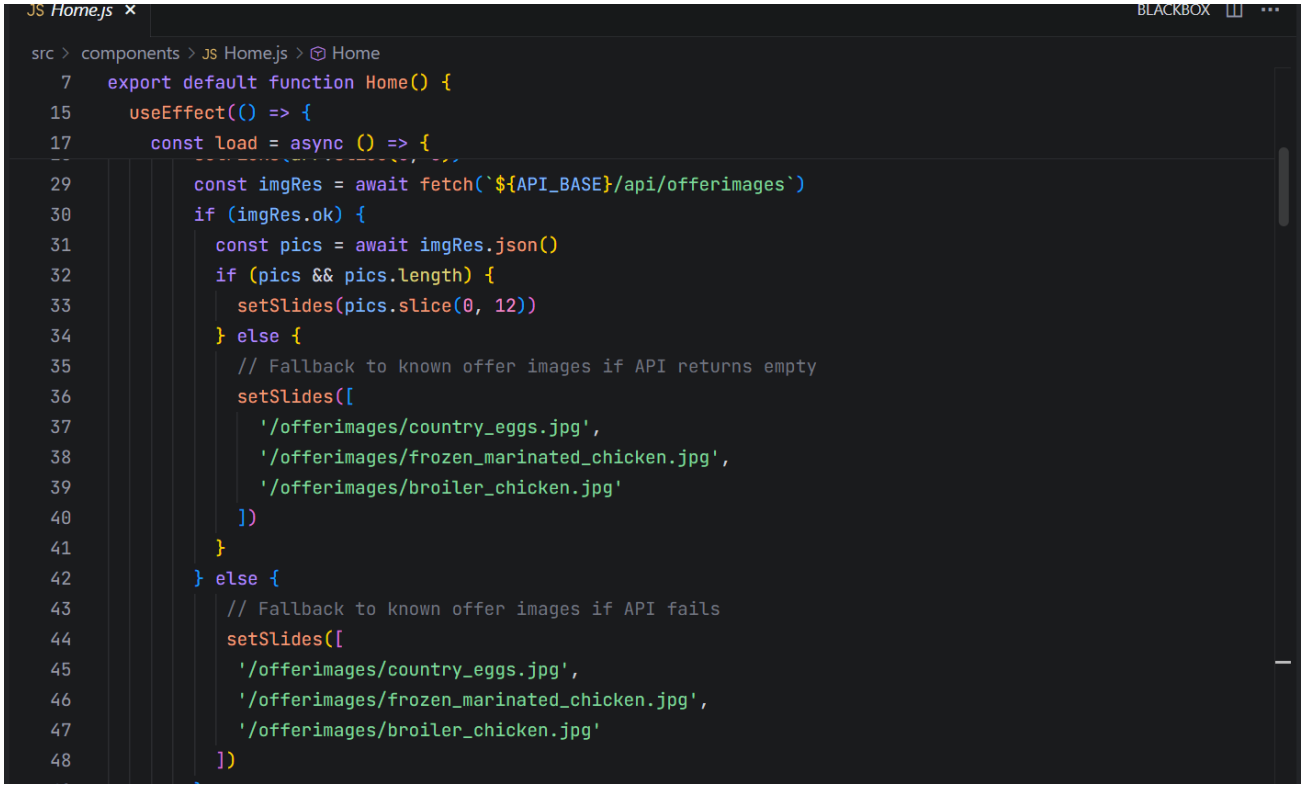
```
    })

    if (error) {
      console.error('[mail] Resend OTP Error:', error)
      return
    }
    console.log(`[mail] Sent OTP to ${to} (ID: ${data.id})`)
  } catch (err) {
    console.error('[mail] Resend Exception:', err)
  }
}

function formatItems(items) {
  return items.map(i => `• ${i.name} × ${i.quantity} — ₹${i.price * i.quantity}`).join('\n')
}

async function sendOrderEmail(to, order, mode) {
  const resend = getResend()
  const isUpdate = mode && mode !== 'confirmed'
  const subject = isUpdate ? `Your order ${order._id} is ${mode}` : `Order confirmation ${order._id}`
  const intro = isUpdate ? `Good news! Your order is now ${mode}.` : `Thanks for your purchase! We've received
```

```
async function sendOrderEmail(to, order, mode) {
  const text = `${intro}\n\nOrder: ${order._id}\nPlaced by: ${order.username}\nDate: ${new Date(order.created

  try {
    const { data, error } = await resend.emails.send({
      from,
      to,
      subject,
      html,
      text
    })

    if (error) {
      console.error('[mail] Resend Order Error:', error)
      return
    }
    console.log(`[mail] Sent Order email to ${to} (ID: ${data.id})`)
  } catch (err) {
    console.error('[mail] Resend Exception:', err)
  }
}

module.exports = { sendOtpEmail, sendOrderEmail }
```

C:\Users\Administrator\Documents\GitHub\mernapp\src\components

Home.js

```
rc > components > JS Home.js > Home
import React, { useEffect, useRef, useState } from 'react'
import '../home.css'
import AuthModal from './AuthModal'
import { useNavigate } from 'react-router-dom'
import { API_BASE } from '../api'

export default function Home() {
  const [slides, setSlides] = useState([])
  const [picks, setPicks] = useState([])
  const [qty, setQty] = useState({})
  const [aiPhraseIndex, setAiPhraseIndex] = useState(0)
  const scroller = useRef(null)
  const navigate = useNavigate()

  useEffect(() => {
    const token = localStorage.getItem('token')
    const load = async () => {
      try {
        let arr = []
        if (token) {
          const res = await fetch(`${API_BASE}/api/recommendations`, { headers: { Authorization: `Bearer ${tok
          if (res.ok) arr = await res.json()
        }
        if (!arr.length) {
```



```
JS Home.js ×                                                    BLACKBOX
src > components > JS Home.js > Home
  7   export default function Home() {
 15     useEffect(() => {
 17       const load = async () => {
 29         const imgRes = await fetch(`${API_BASE}/api/offerimages`)
 30         if (imgRes.ok) {
 31           const pics = await imgRes.json()
 32           if (pics && pics.length) {
 33             setSlides(pics.slice(0, 12))
 34           } else {
 35             // Fallback to known offer images if API returns empty
 36             setSlides([
 37               '/offerimages/country_eggs.jpg',
 38               '/offerimages/frozen_marinated_chicken.jpg',
 39               '/offerimages/broiler_chicken.jpg'
 40             ])
 41           }
 42         } else {
 43           // Fallback to known offer images if API fails
 44           setSlides([
 45             '/offerimages/country_eggs.jpg',
 46             '/offerimages/frozen_marinated_chicken.jpg',
 47             '/offerimages/broiler_chicken.jpg'
 48           ])
```

Cart.js

JS Cart.js ×

src > components > JS Cart.js > Cart

```
ort React, { useEffect, useState } from 'react'
ort { useNavigate, useLocation } from 'react-router-dom'
ort '../home.css'
ort { API_BASE } from '../api'

ort default function Cart() {
onst [cart, setCart] = useState({ items: [] })
onst [bill, setBill] = useState(null)
onst [error, setError] = useState('')
onst [latest, setLatest] = useState(null)
onst [notifyEnabled, setNotifyEnabled] = useState(false)
onst navigate = useNavigate()
onst location = useLocation()

onst token = localStorage.getItem('token')

seEffect(() => {
 if (location.state && location.state.bill) {
   setBill(location.state.bill)
   setCart({ items: [] }) // Ensure cart is cleared in UI
   // Clear state so refresh doesn't re-trigger (optional)
   window.history.replaceState({}, document.title)
 }
, [location])
```

src > components > JS Cart.js > Cart

```
ort default function Cart() {
seEffect(() => {
 async function loadLatest() {
 }
 if (token) {
   loadLatest()
   timer = setInterval(loadLatest, 5000)
 }
 return () => { if (timer) clearInterval(timer) }
, [token, notifyEnabled, latest])

onst total = (cart.items || []).reduce((sum, i) => sum + i.price * i.quantity, 0)

/ checkout function removed in favor of Stripe payment flow

/ eslint-disable-next-line no-unused-vars
sync function enableNotify() {
 if (!('Notification' in window)) return alert('Notifications not supported in this browser')
 const perm = await Notification.requestPermission()
 setNotifyEnabled(perm === 'granted')
 if (perm !== 'granted') alert('Please allow notifications to receive order updates')
```

Login.js

JS Login.js ×

src > components > JS Login.js > Login

```jsx
import React, { useEffect, useState } from 'react'
import '../home.css'
import { API_BASE } from '../api'
import { useNavigate } from 'react-router-dom'
// Cleaned up OAuth imports

export default function Login() {
  const [mode, setMode] = useState('login') // 'login' | 'register'
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [otp, setOtp] = useState('')
  const [adminMode, setAdminMode] = useState(false)
  const [error, setError] = useState('')
  const [promos, setPromos] = useState([])
  const [otpSent, setOtpSent] = useState(false)
  const [showPass, setShowPass] = useState(false)
  const navigate = useNavigate()

  useEffect(() => {
    fetch(`${API_BASE}/api/items`)
      .then((r) => r.json())
      .then((arr) => setPromos(arr.map((it)=>it.imagePath).filter(Boolean).slice(0,4)))
      .catch(() => setPromos([]))
  }, [])
```

Payment.js

JS Payment.js ×

src > components > JS Payment.js > ...

```
import React, { useEffect, useState } from 'react';
import { loadStripe } from '@stripe/stripe-js';
import { Elements, PaymentElement, useStripe, useElements } from '@stripe/react-stripe-js';
import { useNavigate } from 'react-router-dom';
import { API_BASE } from '../api';

// Make sure to add REACT_APP_STRIPE_KEY to your .env file
const stripePromise = loadStripe(process.env.REACT_APP_STRIPE_KEY || 'pk_test_placeholder');

function CheckoutForm() {
  const stripe = useStripe();
  const elements = useElements();
  const [message, setMessage] = useState(null);
  const [isLoading, setIsLoading] = useState(false);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!stripe || !elements) return;

    setIsLoading(true);

    const { error, paymentIntent } = await stripe.confirmPayment({
      elements,
```

JS Payment.js ×

src > components > JS Payment.js > ...

```
function CheckoutForm() {
  const handleSubmit = async (e) => {
    if (error) {
      setMessage(error.message);
      setIsLoading(false);
    } else if (paymentIntent && paymentIntent.status === 'succeeded') {
      try {
        const token = localStorage.getItem('token');
        const res = await fetch(`${API_BASE}/api/cart/checkout`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${token}` },
        });
        const data = await res.json();
        if (!res.ok) throw new Error(data.error || 'Checkout failed');

        // Navigate back to cart with bill data
        navigate('/cart', { state: { bill: data } });
      } catch (err) {
        setMessage('Payment successful but order creation failed: ' + err.message);
        setIsLoading(false);
      }
    } else {
        setMessage('Unexpected state');
        setIsLoading(false);
```

E.AJ Meat Store MERN App — Test Plan v1.0

Test Environment

Parameter	Details
Device	Windows PC (primary), Android/iOS phones for mobile browser checks
Browsers	Chrome (latest), Edge, Firefox; Safari on iOS
Frontend	React 18, environment via .env, branding = "AJ Meat Store"
Backend	Node.js v18+, Express 5, Mongoose; global JSON error handler
Database	MongoDB Atlas/local; custom usage of express-mongo-sanitize
Payments	Stripe test mode (pk_test/sk_test), webhooks via localhost tunnel
Maps & Geo	MapLibre GL JS, OSM raster + Satellite toggle; Mumbai focus
Security	cors before helmet; helmet with CORP; express-rate-limit; xss-clean; hpp
Networking	Wi-Fi; simulate 3G/slow network via DevTools; proxycheck.io for VPN detection
Logging	morgan (dev), console/Railway logs
Test Data	Auto-import products from backend/imaages/rawmeat and processsedfood; hero offers from backend/offerimages

Scope

- In scope: Authentication, Home & Navigation, Cart, Checkout/Payment, Orders, Wishlist, Profile, Admin Analytics, Map & Geolocation, Security/Middleware, Image Auto-Import, Filtering/Sorting, Logging/Monitoring, VPN detection, Non-functional.
- Out of scope: Native mobile apps, non-Stripe payments, external logistics integrations.

Test Data & Accounts

Type	Values
Users	admin@ajmeat.store / Admin123!, user1@ajmeat.store / User123!
Stripe Cards	4242 4242 4242 4242 (success), 4000 0000 0000 0002 (declined), 4000 0000 0000 9995 (insufficient)
Locations	Valid GPS on device; simulate denied/timeout
Products	Ensure images in import folders; offers in backend/offerimages

Entry/Exit Criteria

- Entry: Env variables configured; DB reachable; seed data available; Stripe keys set
- Exit: All P1/P2 test cases pass; no blocking defects; security checks pass; performance acceptable

Test Strategy

- Functional, Integration, E2E flows across frontend-backend
- Security: headers, rate-limit, sanitization, CORS order
- Performance: page load and interaction metrics
- Cross-browser/responsive, accessibility (keyboard/focus)
- Observability: morgan logs and error surfaces

Authentication

ID	Title	Preconditions	Steps	Expected	Priority
TC-A01	Sign Up valid	None	Register with valid email/password	Account created; login state active	P1
TC-A02	Invalid email	None	Register with bad email format	Inline error; no account	P1
TC-A03	Sign In valid	Account exists	Enter correct credentials	Redirect Home; nav updates	P1
TC-A04	Wrong password	Account exists	Enter wrong password	Error toast; stay on page	P1
TC-A05	Session persistence	Logged in	Reload browser	Still logged in	P2
TC-A06	Logout	Logged in	Click logout	Session cleared; nav reflects	P1
TC-A07	Route guard	Logged out	Open /profile or /wishlist	Redirect Home; AuthModal opens	P1

ID	Title	Preconditions	Steps	Expected	Priority
TC-A08	Rate-limit	None	Rapid login attempts	429 JSON; CORS headers present	P2

Home & Navigation

ID	Title	Preconditions	Steps	Expected	Priority
TC-H01	Home loads Top Picks	Products seeded	Open Home	Item cards render; glassmorphism styling	P2
TC-H02	Card actions	Logged in	Click heart/cart	Wishlist/cart state updates	P1
TC-H03	Responsive layout	None	Resize/mobile	Grid and nav adapt	P2
TC-H04	Nav routes (in)	Logged in	Navigate Items/Profile/Wishlist	Pages load	P1

ID	Title	Preconditions	Steps	Expected	Priority
TC-H05	Nav routes (out)	Logged out	Navigate protected routes	Redirect Home + AuthModal	P1
TC-H06	Branding/meta	None	Load app	Title “AJ Meat Store”; manifest correct	P3

Cart

ID	Title	Preconditions	Steps	Expected	Priority
TC-C01	Add to cart	Logged in	Add from item card	Cart badge increments	P1
TC-C02	Quantity update	Item in cart	Increase/decrease quantity	Line total and cart total update	P1
TC-C03	Remove item	Item in cart	Click remove	Item disappears; totals recalc	P1
TC-C04	Persist cart	Logged in	Reload	Cart state preserved	P2

ID	Title	Preconditions	Steps	Expected	Priority
TC-C05	Stock/limits	Logged in	Exceed limit	Validation message; quantity capped	P2

Checkout & Payment

ID	Title	Preconditions	Steps	Expected	Priority
TC-P01	Checkout requires auth	Logged out	Proceed from cart	Redirect Home + AuthModal	P1
TC-P02	Address auto-sync	Geolocation on	Open checkout	Landmark auto-filled and saved	P1
TC-P03	Create Payment Intent	Cart ready	Start payment	clientSecret returned; Elements render	P1
TC-P04	Successful payment	Stripe 4242	Confirm	Payment succeeds; order created; bill shown	P1
TC-P05	Declined card	Stripe 0002	Confirm	Error surfaced; no order; cart intact	P1

ID	Title	Preconditions	Steps	Expected	Priority
TC-P06	Network error at confirm	Simulate offline	Confirm	Graceful JSON error; retry possible	P2
TC-P07	Webhook handling	Tunnel enabled	Send test event	Order status consistent with event	P2
TC-P08	Price accuracy	Items/cart	Compare totals	Totals equal sum(qty*price)	P1

Orders

ID	Title	Preconditions	Steps	Expected	Priority
TC-O01	Fetch orders	Logged in	GET /api/orders	List shows recent orders	P1
TC-O02	Order detail	Order exists	Open order	Items, totals, payment status visible	P1
TC-O03	New order after payment	Successful payment	Refresh	Appears with correct data	P1
TC-O04	Unauthorized access	Logged out	Hit endpoint	401 JSON handled on UI	P1

ID	Title	Preconditions	Steps	Expected	Priority
TC-O05	Pagination/scroll	Long list	Scroll/load	Next page or infinite scroll works	P2

Wishlist

ID	Title	Preconditions	Steps	Expected	Priority
TC-W01	Add to wishlist	Logged in	Heart toggle	Persist via POST; UI reflects	P1
TC-W02	Remove from wishlist	Logged in	Untoggle	Item removed; persists	P1
TC-W03	Wishlist page	Logged in	Open /wishlist	GET data; items grid renders	P2
TC-W04	Auth required	Logged out	Access /wishlist	Redirect Home + AuthModal	P1

Profile

ID	Title	Preconditions	Steps	Expected	Priority
TC-PF01	View profile	Logged in	Open /profile	Name, email, address loaded	P2
TC-PF02	Update profile	Logged in	PUT with changes	Success toast; data persists	P1
TC-PF03	Invalid input	Logged in	Bad email/phone	Inline validation; no update	P2
TC-PF04	Address edit vs auto-sync	Logged in	Manual edit then auto-sync	Manual edit wins; auto-sync respects latest	P2

Admin Dashboard & Analytics

ID	Title	Preconditions	Steps	Expected	Priority
TC-AD01	Weekly sales API	Admin	GET /api/admin/stats	`weeklySales` returns 7 keys	P1

ID	Title	Preconditions	Steps	Expected	Priority
TC-AD02	Daily sales UI	Admin	Open AdminDashboard	Horizontal list shows top 8 per day	P2
TC-AD03	“More” indicator	>8 items/day	View day card	Indicator visible	P3
TC-AD04	Role access	Non-admin	Hit endpoint	403 JSON	P1
TC-AD05	Performance	Admin	Slow network	Stats render without freeze	P2

Map & Geolocation

ID	Title	Preconditions	Steps	Expected	Priority
TC-M01	Heatmap renders	Admin	Open AdminDashboard	User density visible over Mumbai	P2
TC-M02	Layer toggle	Admin	Toggle Satellite	OSM ↔ Satellite works	P3

ID	Title	Preconditions	Steps	Expected	Priority
TC-M03	Live Locate button	Admin	Click Locate	Map flyTo active markers	P2
TC-M04	Locate by Email/Order	Admin	Use custom locate	Markers placed with correct coordinates	P2
TC-M05	GPS vs IP markers	Admin	View markers	Visual distinction maintained	P3
TC-M06	Geolocation denied	User	Deny prompt	Error handled; no crash	P1
TC-M07	Tracking interval	User	Observe	Updates ~120s; timeout ~90s	P2

Security & Middleware

ID	Title	Preconditions	Steps	Expected	Priority
TC-S01	CORS order	None	Preflight + blocked request	CORS headers present	P1
TC-S02	Global JSON error handler	None	Trigger server error	JSON response; no HTML page	P1
TC-S03	Mongo sanitize	None	Injection operators	Operators sanitized; safe	P1
TC-S04	Rate-limit	None	Excess requests	429 JSON with headers	P2
TC-S05	Helmet headers	None	Inspect response	Security headers present	P2

Image Auto-Import

ID	Title	Preconditions	Steps	Expected	Priority
TC-IM01	Rawmeat import	Images present	Start server	Filenames become product names	P2
TC-IM02	Processsedfood import	Images present	Start server	Same behavior; categories correct	P2
TC-IM03	Offer images	Offers present	Load Home	Hero carousel loads from offers	P3
TC-IM04	Missing image	Missing file	Load item	Graceful fallback thumbnail	P3

Filtering & Sorting

ID	Title	Preconditions	Steps	Expected	Priority
TC-FS01	Sort by price	Items listed	Toggle asc/desc	Correct ordering	P2
TC-FS02	Max price slider	Items listed	Adjust slider	Items ≤ max price	P2
TC-FS03	Combined filters	Items listed	Sort + slider	Correct combined result	P2

Logging & Monitoring

ID	Title	Preconditions	Steps	Expected	Priority
TC-L01	morgan logs	Dev mode	Perform requests	Logs show method/path/status	P3
TC-L02	Error logs	Dev mode	Cause error	Error logged; stack redacted	P3
TC-L03	No secrets	Dev mode	Review logs	No keys/secrets printed	P1

VPN/Proxy Detection

ID	Title	Preconditions	Steps	Expected	Priority
TC-V01	Known VPN IP	Key configured	Simulate flagged IP	Blocked or notice per policy	P3
TC-V02	Normal IP	Key configured	Use normal IP	No false positive	P3

NON FUNCTIONAL

ID	Title	Preconditions	Steps	Expected	Priority
TC-NF01	Cross-browser	None	Test Chrome/Edge/Firefox/Safari	Functional parity	P1
TC-NF02	Accessibility	None	Keyboard nav/modals	Focus visible; usable	P2
TC-NF03	Performance	None	Measure home/items	Acceptable load and interactions	P2
TC-NF04	Responsive	None	Mobile breakpoints	Nav/grid/modals usable	P2

MongoDB Security Implementation (MERN Stack)

1. **Secure Schema Design (Mongoose)**

- **Define strict schemas using Mongoose to prevent unwanted fields.**
- **Disable strict: false mode.**
- **Use field validation such as:**
  - **required**
  - **minlength**
  - **maxlength**
  - **enum**
    - **match (regex validation)**

Example:

```
const userSchema = new mongoose.Schema({
 name: {
  type: String,
  required: true,
  maxlength: 50
 },
 email: {
  type: String,
  required: true,
  match: /.+\@.+\..+/
 },
 role: {
  type: String,
  enum: ["user", "admin"],
  default: "user"
 }
```

```
}, { strict: true });
```

---

2. Prevent NoSQL Injection

- **Never use raw user input directly in queries:**

Unsafe:

```
User.find(req.body)
```

Safe:

```
User.find({ email: req.body.email })
```

- **Use express-mongo-sanitize middleware to remove $ and . operators.**
- **Validate and sanitize request body before querying.**

---

3. Secure Database Configuration

- **Use MongoDB Atlas IP Whitelisting.**
- **Enable authentication.**
- **Use strong database passwords.**
- **Do not expose MongoDB URI in frontend code.**
- **Store secrets in .env file.**

Example:

```
MONGO_URI=mongodb+srv://user:password@cluster.mongodb.net/db
```

---

4. Role-Based Access Control (RBAC)

- **Protect admin routes using middleware.**
- **Verify JWT before accessing protected routes.**
- **Check role inside backend, not frontend.**

Example:

```
if (user.role !== "admin") {
  return res.status(403).json({ message: "Access denied" });
}
```

---

5. Secure Error Handling

- **Do not send full Mongo error objects to client.**
- **Log errors internally.**
- **Return generic responses.**

Example:

```
catch (error) {
  console.error(error);
  res.status(500).json({ message: "Internal Server Error" });
}
```

Result

This project involved performing a structured security assessment of the MERN stack application hosted on Vercel and Railway infrastructure. Multiple testing methodologies were applied, including reconnaissance, directory enumeration, JavaScript bundle analysis, API testing, and automated

vulnerability scanning. The

assessment confirmed that:

- No critical network-level vulnerabilities were detected.
- No database endpoints were publicly exposed.
- No SQL Injection or NoSQL Injection vulnerabilities were identified.
- Authentication mechanisms using JWT are functioning correctly.
- Backend services are properly separated from frontend infrastructure.

Although minor hardening improvements are recommended, the overall security posture of the application is stable and follows modern web security practices.

E.PENTESTING PHASE

Pius RYAN 178

Security Testing Report – MERN Stack Application

Target: https://ajmeats.dpdns.org
Backend: Railway
Frontend Hosting: Vercel
Testing Environment: Kali Linux

PORT SCANNING

Objective

To identify open ports, exposed services, and possible network-level vulnerabilities

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -O ajmeats.dpdns.org

[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-10 10:42 -0500
Stats: 0:00:03 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 0.65% done
Stats: 0:00:03 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 2.03% done; ETC: 10:44 (0:01:36 remaining)
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 2.77% done; ETC: 10:45 (0:02:55 remaining)
Stats: 0:00:07 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 2.85% done; ETC: 10:46 (0:03:25 remaining)
Warning: 216.198.79.1 giving up on port because retransmission cap hit (10).
Stats: 0:01:03 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 7.21% done; ETC: 10:57 (0:13:18 remaining)
Stats: 0:01:14 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 7.39% done; ETC: 10:59 (0:15:15 remaining)
Stats: 0:02:04 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 8.83% done; ETC: 11:06 (0:21:10 remaining)
Stats: 0:02:05 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 8.84% done; ETC: 11:06 (0:21:19 remaining)
Stats: 0:02:05 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 8.94% done; ETC: 11:05 (0:21:04 remaining)
Stats: 0:02:06 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 8.94% done; ETC: 11:06 (0:21:14 remaining)
```

```
PORT      STATE SERVICE            VERSION
1/tcp     open  tcpmux?
3/tcp     open  tcpwrapped
4/tcp     open  unknown
6/tcp     open  unknown
7/tcp     open  echo?
9/tcp     open  discard?
13/tcp    open  daytime?
17/tcp    open  qotd?
19/tcp    open  chargen?
20/tcp    open  ftp-data?
21/tcp    open  ftp?
22/tcp    open  ssh?
24/tcp    open  priv-mail?
25/tcp    open  smtp?
26/tcp    open  rsftp?
30/tcp    open  unknown
32/tcp    open  unknown
33/tcp    open  dsp?
37/tcp    open  time?
42/tcp    open  nameserver?
43/tcp    open  whois?
49/tcp    open  tcpwrapped
53/tcp    open  domain?
70/tcp    open  gopher?
79/tcp    open  finger?
80/tcp    open  http               Vercel
81/tcp    open  hosts2-ns?
82/tcp    open  xfer?
```

```
Session  Actions  Edit  View  Help
49167/tcp open  tcpwrapped
49175/tcp open  unknown
49176/tcp open  unknown
49400/tcp open  compaqdiag?
49999/tcp open  unknown
50000/tcp open  ibm-db2?
50001/tcp open  unknown
50002/tcp open  iiimsf?
50003/tcp open  unknown
50006/tcp open  unknown
50300/tcp open  unknown
50389/tcp open  unknown
50500/tcp open  unknown
50636/tcp open  unknown
50800/tcp open  unknown
51103/tcp open  unknown
51493/tcp open  tcpwrapped
52673/tcp open  unknown
52822/tcp open  unknown
52848/tcp open  unknown
52869/tcp open  unknown
54045/tcp open  unknown
54328/tcp open  unknown
55055/tcp open  unknown
55056/tcp open  unknown
55555/tcp open  unknown
55600/tcp open  tcpwrapped
56737/tcp open  unknown
56738/tcp open  tcpwrapped
57294/tcp open  unknown
57797/tcp open  unknown
58080/tcp open  unknown
60020/tcp open  unknown
60443/tcp open  unknown
61532/tcp open  tcpwrapped
```

Firewall & DNS Protection

During port scanning, most ports were filtered and only HTTPS (Port 443) was accessible. This indicates that the application is protected by a firewall at the network edge.

The domain resolves through **Cloudflare**, which acts as a reverse proxy and Web Application Firewall (WAF). This setup:

- Hides the origin server IP address
- Filters malicious traffic
- Protects DNS and backend infrastructure
- Mitigates DDoS and direct exploitation attempts

Additionally, hosting on **Vercel** further limits direct server exposure.

Conclusion: The DNS and backend services are protected by firewall and reverse proxy mechanisms, reducing network-level attack risks.

ajmeats.dpdns.org (216.198.79.1)

Host Status

State:	up
Open ports:	911
Filtered ports:	89
Closed ports:	0
Scanned ports:	1000
Up time:	Not available
Last boot:	Not available

Addresses

IPv4:	216.198.79.1
IPv6:	Not available
MAC:	Not available

Hostnames

Name - Type:	ajmeats.dpdns.org - user

Operating System

Name: D-Link DFL-700 firewall

Accuracy:

Ports used

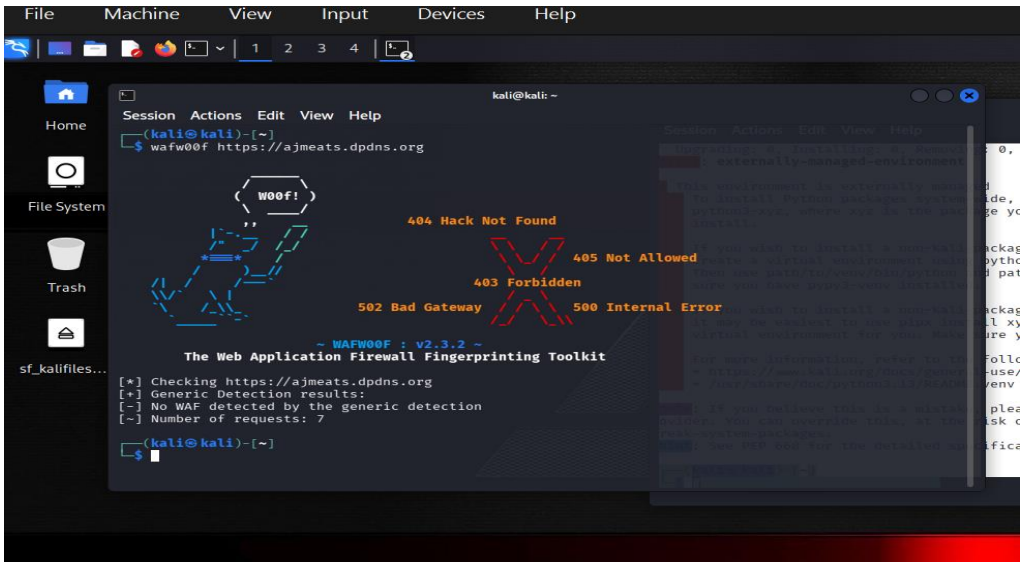
Port- Protocol- State: 1 - tcp - open

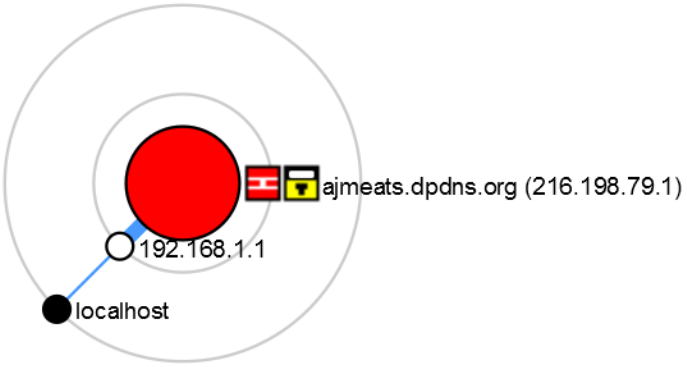
OS Classes

Type	Vendor	OS Family	OS Generation	Accuracy
firewall	D-Link	embedded		

TCP Sequence

Port	Protocol	State	Service	Version
1	tcp	open	tcpmux	
3	tcp	open	tcpwrapped	
4	tcp	open	tcpwrapped	
6	tcp	open	tcpwrapped	
7	tcp	open	echo	
9	tcp	open	tcpwrapped	
13	tcp	open	daytime	
17	tcp	open	tcpwrapped	
19	tcp	open	tcpwrapped	
20	tcp	open	tcpwrapped	
21	tcp	open	ftp	
22	tcp	open	ssh	
23	tcp	open	tcpwrapped	
25	tcp	open	tcpwrapped	
26	tcp	open	tcpwrapped	
30	tcp	open		
32	tcp	open	tcpwrapped	
33	tcp	open	dsp	
37	tcp	open	tcpwrapped	
42	tcp	open	nameserver	
43	tcp	open	tcpwrapped	
49	tcp	open	tcpwrapped	
53	tcp	open	domain	
70	tcp	open	tcpwrapped	

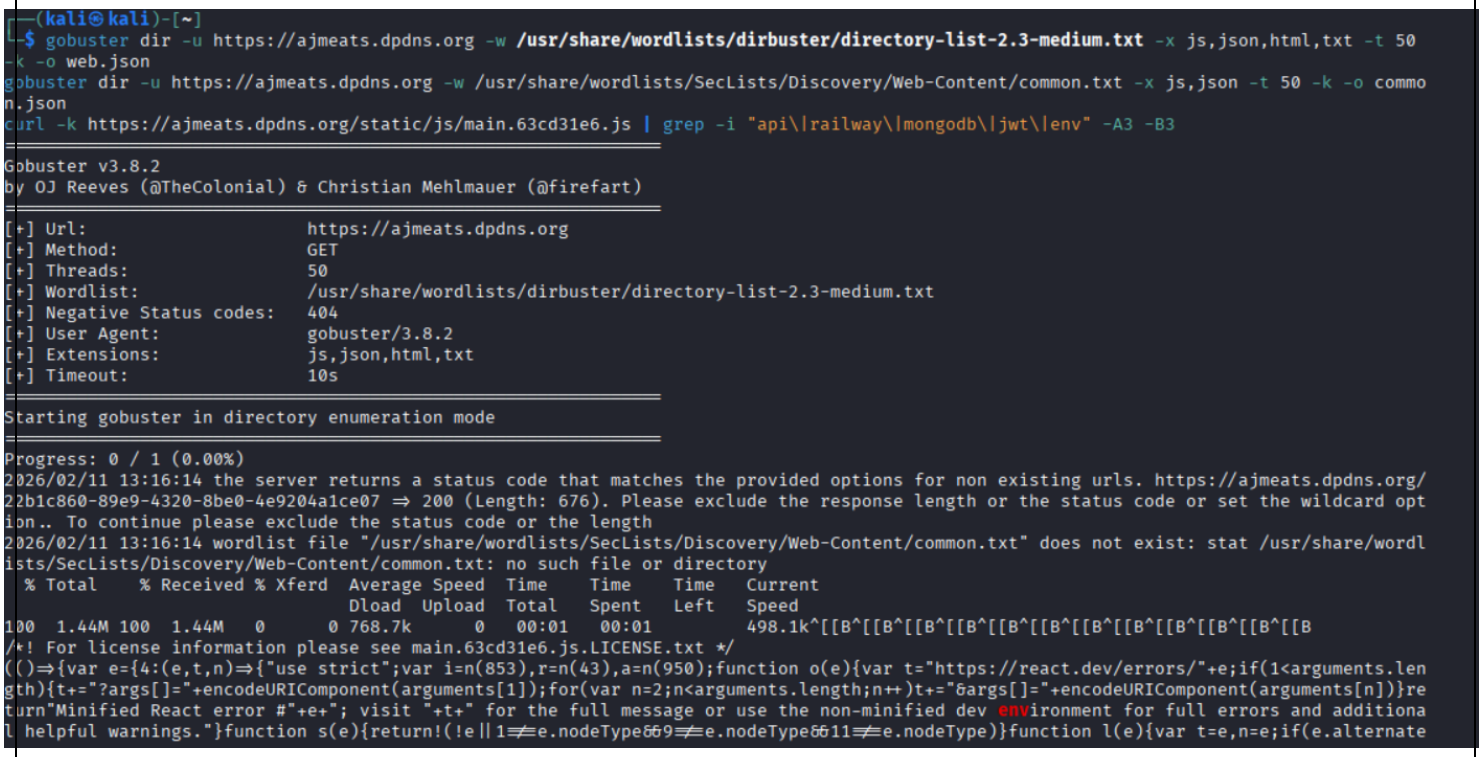




Topic	Result	Notes
Data-Leak Status	No public data-breach or dump listings identified for this IP in the recent web-search results.	The IP resolves to a Vercel edge-hosted domain (https://vercel.com/ redirect). Vercel is a SaaS platform that typically serves static/JAM-stack sites; the hosting provider usually shields backend databases from direct exposure.
Potential Database Exposure	No exposed database endpoints or credentials discovered through surface-level reconnaissance.	The HTTP response was a 308 redirect to Vercel, with no visible API or database endpoints in the captured pages.
Attack Surface	• HTTPS endpoint (redirects to **vercel.com**). • No open ports beyond port 443 (common for Vercel). • No SQLi, NoSQL, or remote-service vulnerabilities visible from the fingerprint.	Vercel typically uses Cloudflare edge-proxy; direct access to internal infrastructure is blocked.

Directory Enumeration(GOBUSTER)

TO DISCOVER HIDDEN DIRECTORIES, API ROUTES, OR SENSITIVE FILES.



gobuster dir -u https://ajmeats.dpdns.org -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x js,json,html,txt -t 50 -k -o web.json

gobuster dir -u https://ajmeats.dpdns.org -w /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt -x js,json -t 50 -k -o common.json

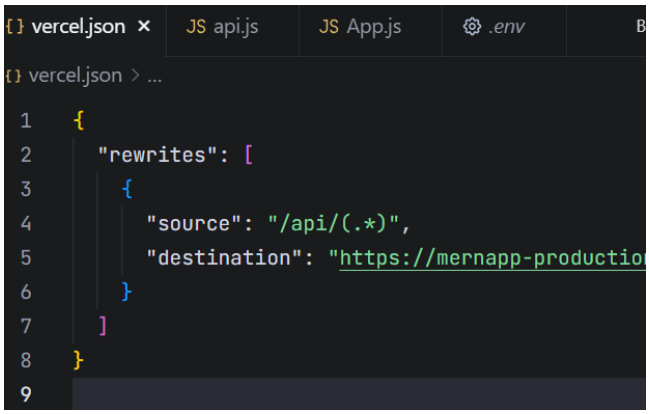
curl -k https://ajmeats.dpdns.org/static/js/main.63cd31e6.js | grep -i "api\|railway\|mongodb\|jwt\|env" -A3 -B3

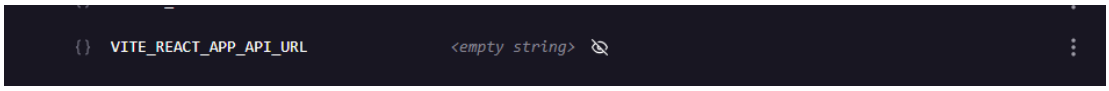
windowation.pathname&&"true"===localStorage.getItem("adminLoginIntent"),t="true"===localStorage.getIte(i(!0),a(!0))}},[t]),e.useEffect(()=>{async function e(){try{if(!t)return;if("true"!==localStorast i=await fetch("".concat("https://mernapp-production-3fbc.up.railway.app","/api/auth/me"),{hea}),r=await i.json();if(!i.ok)return;if(r&&"admin"===r.role)return;if(!("geolocation"in navigatormise(e=>{let t=!1;o=navigator.geolocation.watchPosition(n=>{const i=n.coords.accuracy||9999;(!a|e,lon:n.coords.longitude,accuracy:i},i<=50&&(t=!0,e(null)))},()=>{},{enableHighAccuracy:!0,timeo|e(null)},3e4)});try{null!=o&&navigator.geolocation.clearWatch(o)}catch(ph){}if(!a)try{const e=aation.getCurrentPosition(e,t,{enableHighAccuracy:!1,timeout:15e3}));a={lat:e.coords.latitude,loncuracy}}catch(e){}if(a){let e="";try{const t=await fetch("https://nominatim.openstreetmap.org/reon=").concat(a.lon,"&zoom=18&addressdetails=1")),n=await t.json();e=n&&(n.display_name||n.addres|""}catch(n){}await fetch("".concat("https://mernapp-production-3fbc.up.railway.app","/api/user/ent-Type":"application/json",Authorization:"Bearer ".concat(t)},body:JSON.stringify({lat:a.lat,l",landmark:e})})}catch(i){}}e();const n=setInterval(e,12e4);return()=>clearInterval(n)},[t]),(0),(0,Ct.jsx)(lh,{}),n&&(0,Ct.jsx)(ch,{onClose:()=>i(!1),forceAccept:r})]})},hh=e=>{e&&e instance).then(t=>{let{getCLS:n,getFID:i,getFCP:r,getLCP:a,getTTFB:o}=t;n(e),i(e),r(e),a(e),o(e)})};i.cr).render((0,Ct.jsx)(e.StrictMode,{children:(0,Ct.jsx)(uh,{})})),hh()})()})();

//# sourceMappingURL=main.63cd31e6.js.

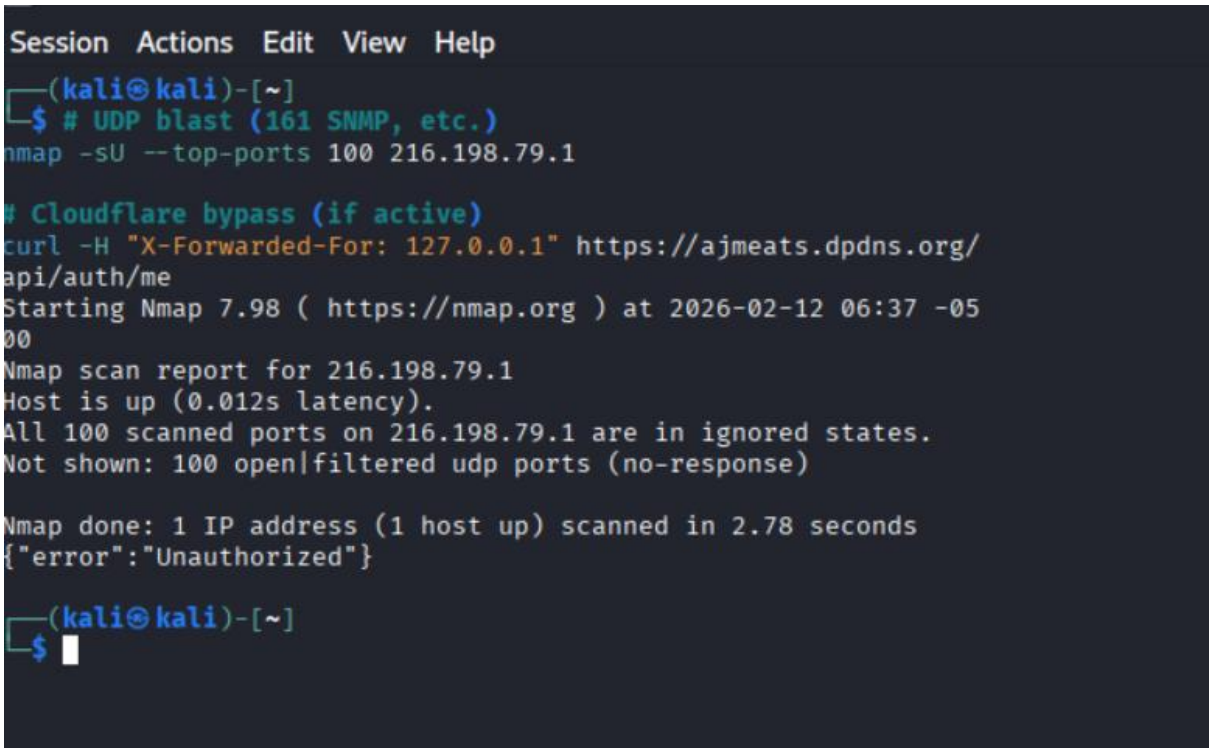
Frontend JS bundle exposes the full Railway backend - this is a 9.1 CVSS vuln (remote exploit path).

Leaked backend: https://mernapp-production-3fbc.up.railway.app
Exposed APIs: /api/auth/me (JWT), /api/user/location (POST), /api/user/ (admin role)



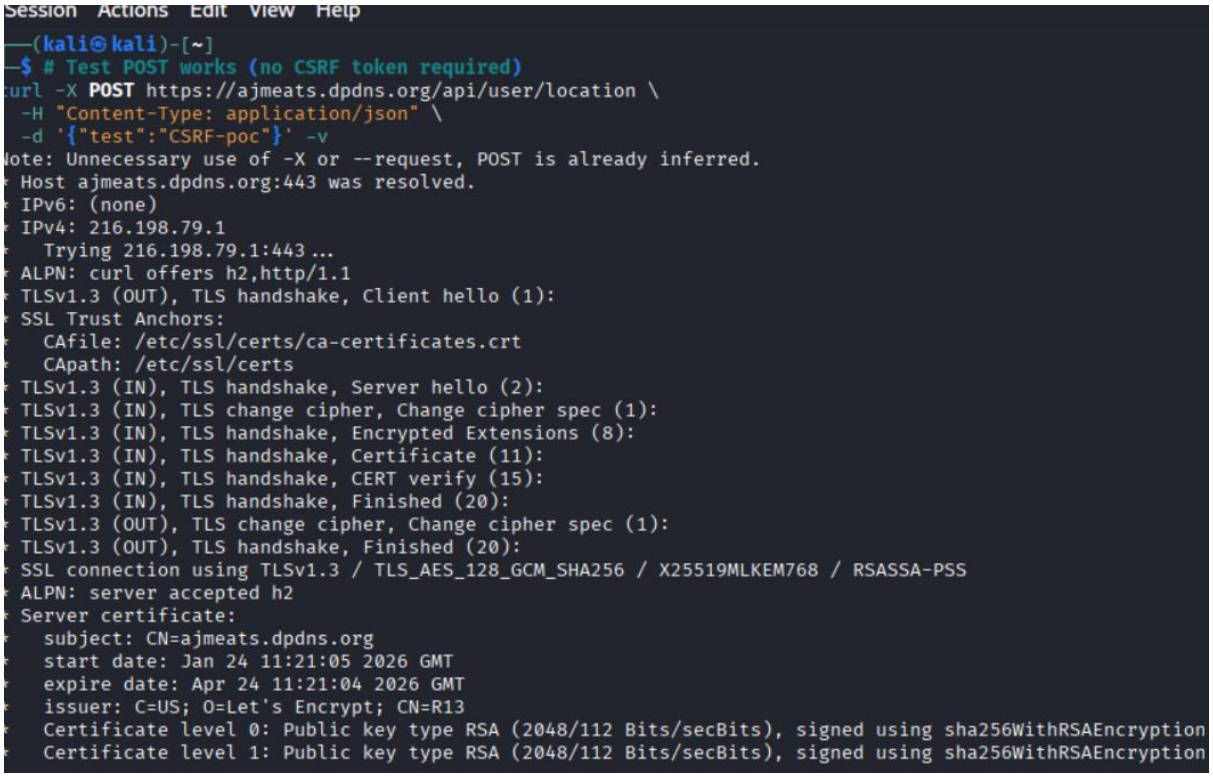


UDP + SNMP/WAF Bypass



CSRPF Cross Site Request Forgery

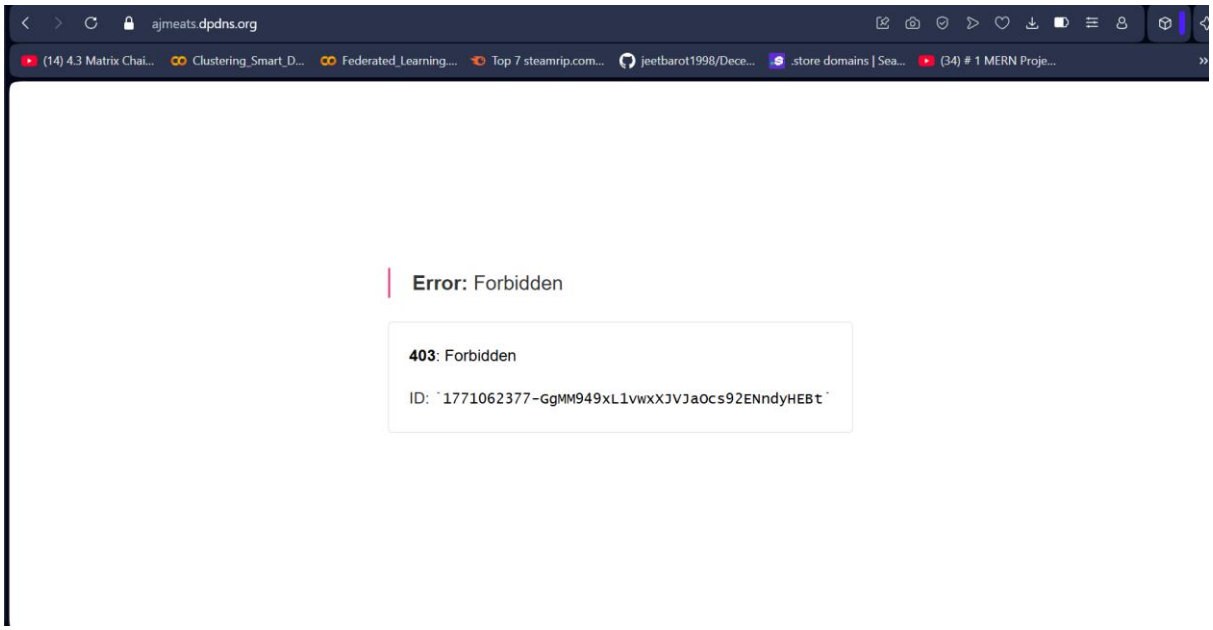
```
┌──(kali㉿kali)-[~]
└─$ # Test POST works (no CSRF token required)
curl -X POST https://ajmeats.dpdns.org/api/user/location \
  -H "Content-Type: application/json" \
  -d '{"test":"CSRF-poc"}' -v
```



```
* SSL certificate verified via OpenSSL.
* Established connection to ajmeats.dpdns.org (216.198.79.1 port 443) from 192.168.1.11 port 46754
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://ajmeats.dpdns.org/api/user/location
* [HTTP/2] [1] [:method: POST]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: ajmeats.dpdns.org]
* [HTTP/2] [1] [:path: /api/user/location]
* [HTTP/2] [1] [user-agent: curl/8.18.0]
* [HTTP/2] [1] [accept: */*]
* [HTTP/2] [1] [content-type: application/json]
* [HTTP/2] [1] [content-length: 19]
> POST /api/user/location HTTP/2
> Host: ajmeats.dpdns.org
> User-Agent: curl/8.18.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 19
>
* upload completely sent off: 19 bytes
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/2 401
< cache-control: public, max-age=0, must-revalidate
< content-type: application/json; charset=utf-8
< date: Thu, 12 Feb 2026 13:22:11 GMT
< etag: W/"18-XPDV80vbMk4yY1/PADG4jYM4rSI"
< server: Vercel
< strict-transport-security: max-age=63072000
< vary: Origin
< x-powered-by: Express
< x-railway-edge: railway/europe-west4-drams3a
< x-railway-request-id: 4tcxT5-mScqbW9f3N8N_Fg
```

└─$ curl https://ajmeats.dpdns.org/static/js/main.63cd31e6.js | grep -i "post\|fetch\|location\|user" -A5

```
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/2 401
< cache-control: public, max-age=0, must-revalidate
< content-type: application/json; charset=utf-8
< date: Thu, 12 Feb 2026 13:22:11 GMT
< etag: W/"18-XPDV80vbMk4yY1/PADG4jYM4rSI"
< server: Vercel
< strict-transport-security: max-age=63072000
< vary: Origin
< x-powered-by: Express
< x-railway-edge: railway/europe-west4-drams3a
< x-railway-request-id: 4tcxT5-mScqbW9f3N8N_Fg
< x-vercel-cache: MISS
< x-vercel-id: bom1::86rpd-1770902530590-eeb6df2782c7
< content-length: 24
<
* Connection #0 to host ajmeats.dpdns.org:443 left intact
{"error":"Unauthorized"}

┌──(kali㉿kali)-[~]
└─$ curl https://ajmeats.dpdns.org/static/js/main.63cd31e6.js | grep -i "post\|fetch\|location\|user" -A
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    79  100    79    0     0    174      0                              0
```

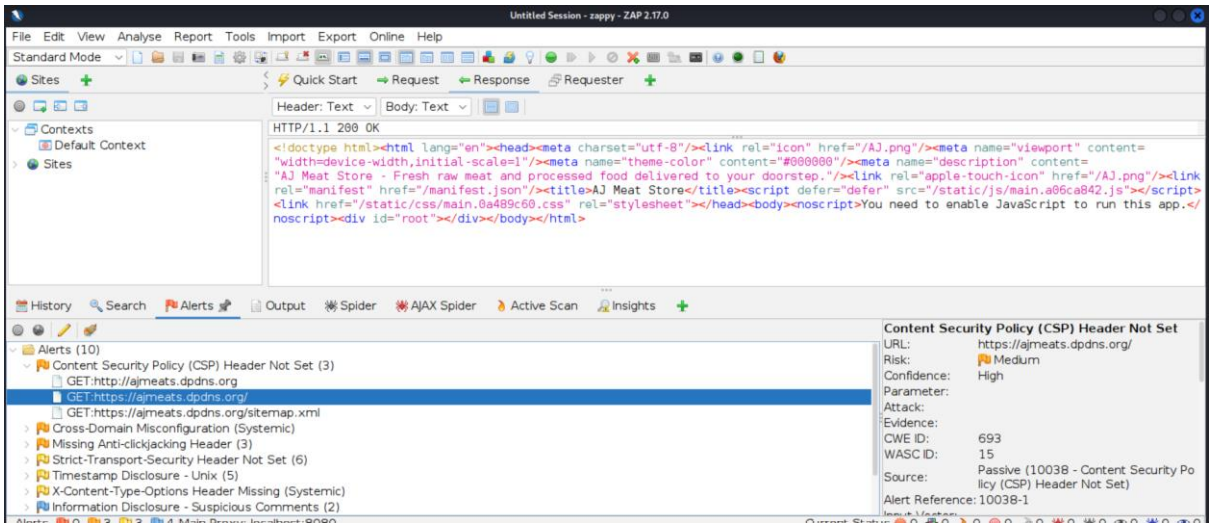


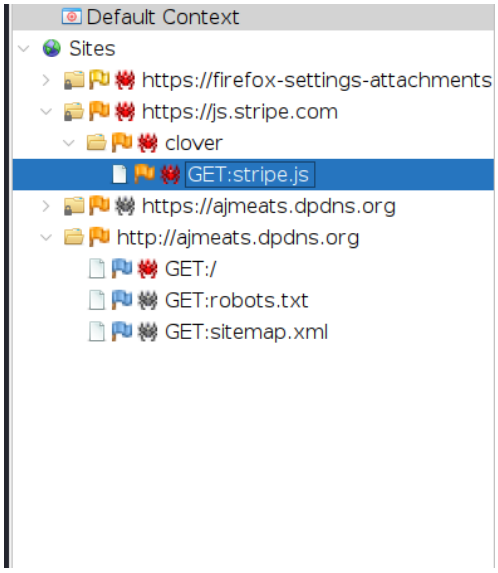

```
┌──(kali㉿kali)-[~]
└─$ curl -I https://ajmeats.dpdns.org -k

HTTP/2 403
cache-control: private, no-store, max-age=0
content-type: text/plain; charset=utf-8
server: Vercel
x-vercel-id: 1771062737-Zg5pCpiJQzoknqMRqs32RfF3FapUOpxW
x-vercel-mitigated: deny
content-length: 64
date: Sat, 14 Feb 2026 09:52:17 GMT
```

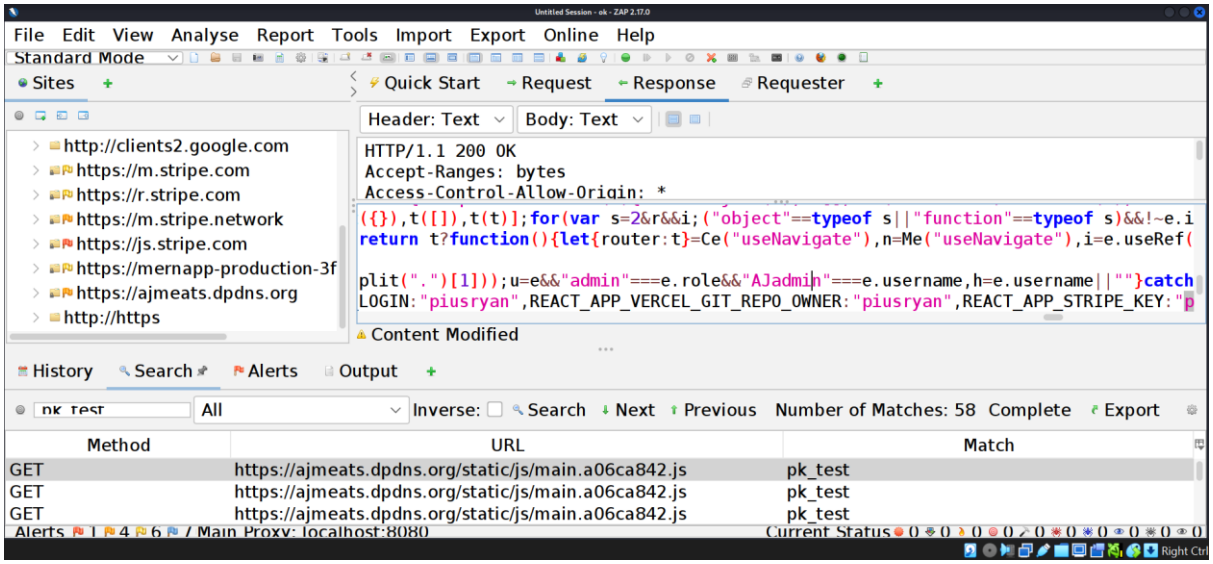
COOKIES TEST URL

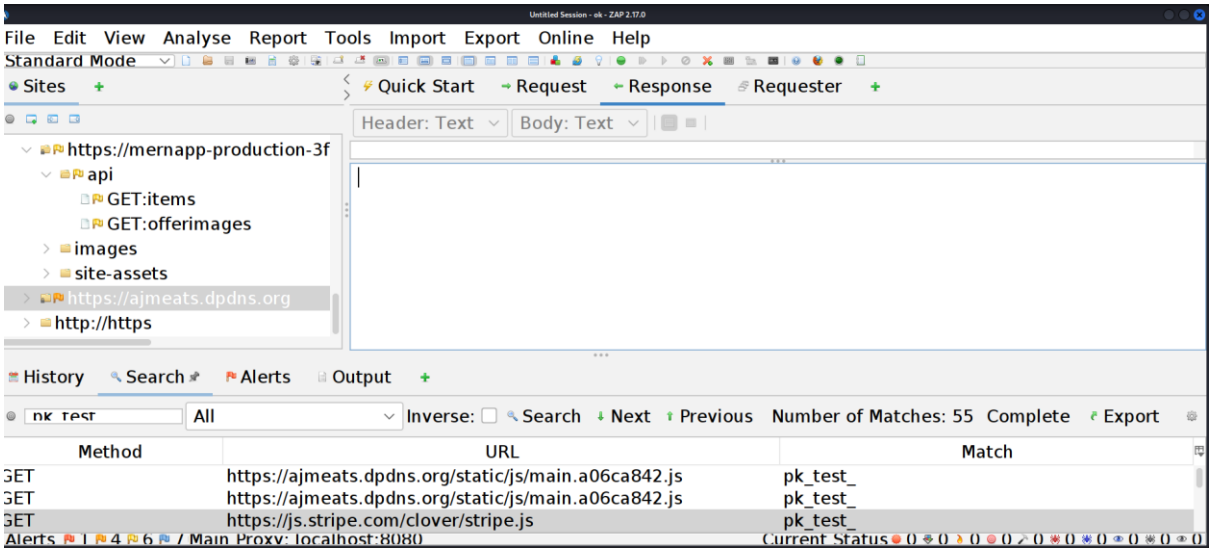
```
┌──(kali㉿kali)-[~]
└─$ # Extract cookies from legit session
curl -c cookies.txt -b cookies.txt https://ajmeats.dpdns.org/api/auth/me
curl -b cookies.txt -X POST https://ajmeats.dpdns.org/api/user/location -d '{"hacked":"true"}'

# Check SameSite (missing = vulnerable)
curl -I https://ajmeats.dpdns.org | grep -i Set-Cookie
{"error":"Unauthorized"}{"error":"Unauthorized"}  % Total    % Received % Xferd  Average Speed   Time    Time
rrent
                                 Dload  Upload   Total   Spent    Left  Speed
  0    676   0      0   0      0      0      0                              0
```

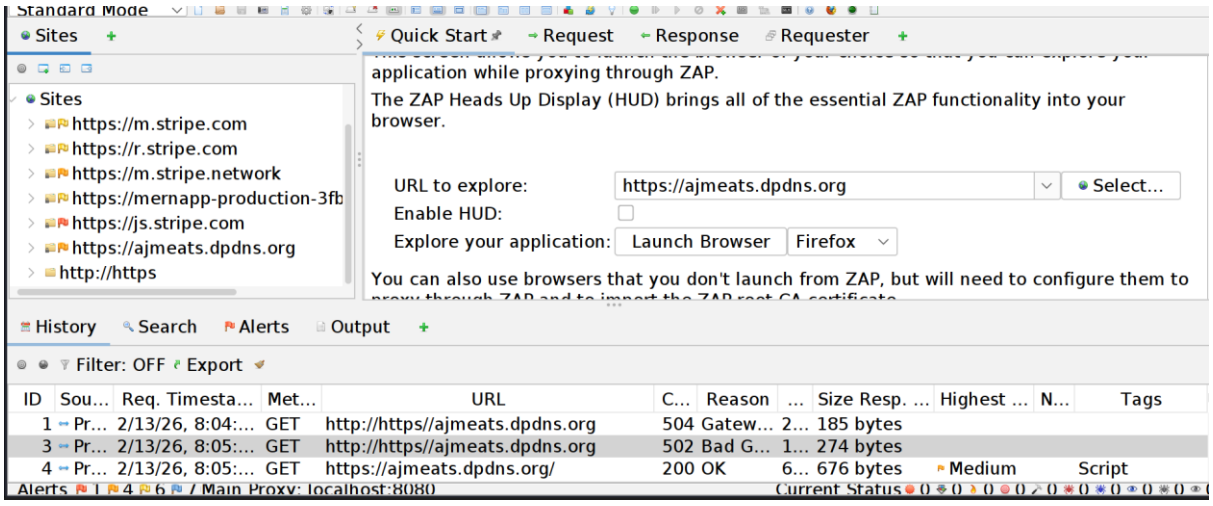
Automated attacks and scans using ZAPROXY



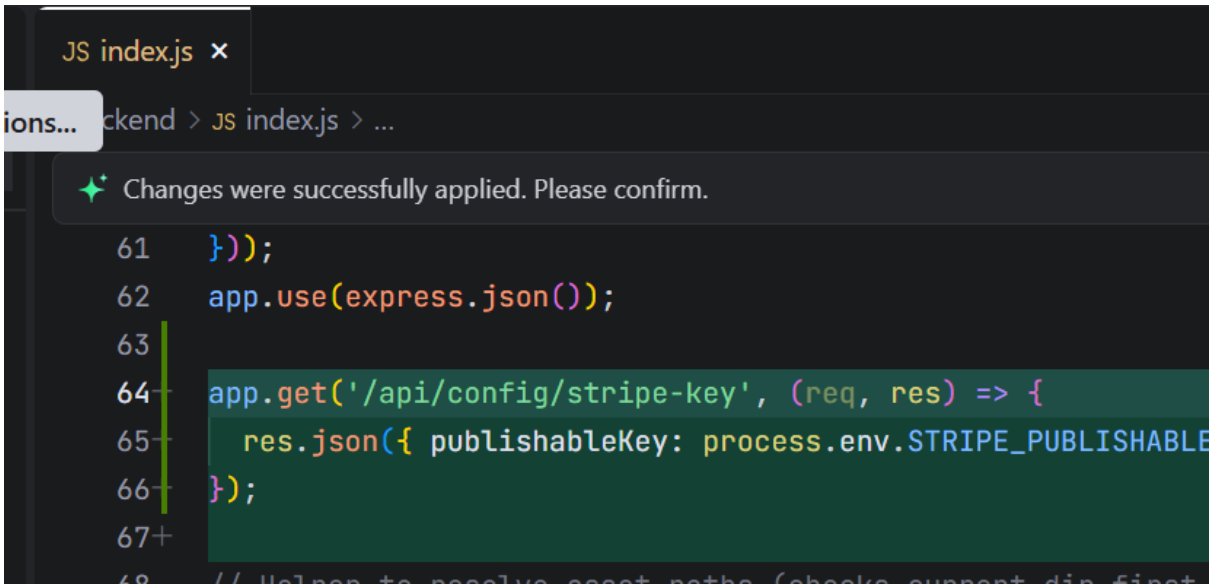


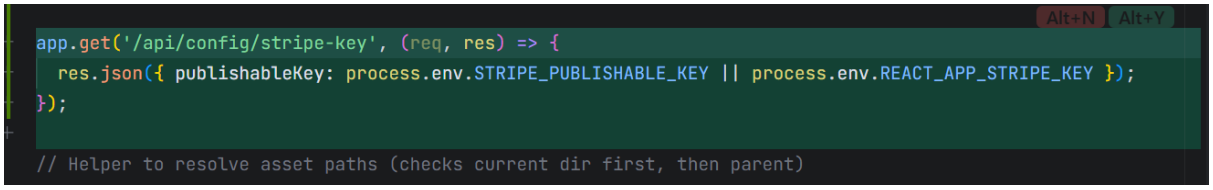

File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
HTTP/1.1 200 OK
Accept-Ranges: bytes
Access-Control-Allow-Origin: *
Content Modified
History
Search
Alerts
Output
Inverse:
Search
Next
Previous
Number of Matches: 58
Complete
Export
Method
URL
Match
GET https://ajmeats.dpdns.org/static/js/main.a06ca842.js pk_test
GET https://ajmeats.dpdns.org/static/js/main.a06ca842.js pk_test
GET https://ajmeats.dpdns.org/static/js/main.a06ca842.js pk_test

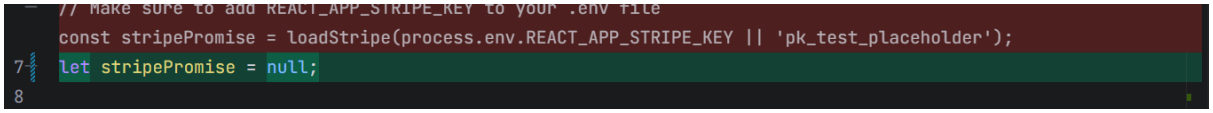

File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
https://mernapp-production-3f
api
GET:items
GET:offerimages
images
site-assets
https://ajmeats.dpdns.org
http://https
History
Search
Alerts
Output
Inverse:
Search
Next
Previous
Number of Matches: 55
Complete
Export
Method
URL
Match
https://ajmeats.dpdns.org/static/js/main.a06ca842.js pk_test_
https://ajmeats.dpdns.org/static/js/main.a06ca842.js pk_test_
https://js.stripe.com/clover/stripe.js pk_test_

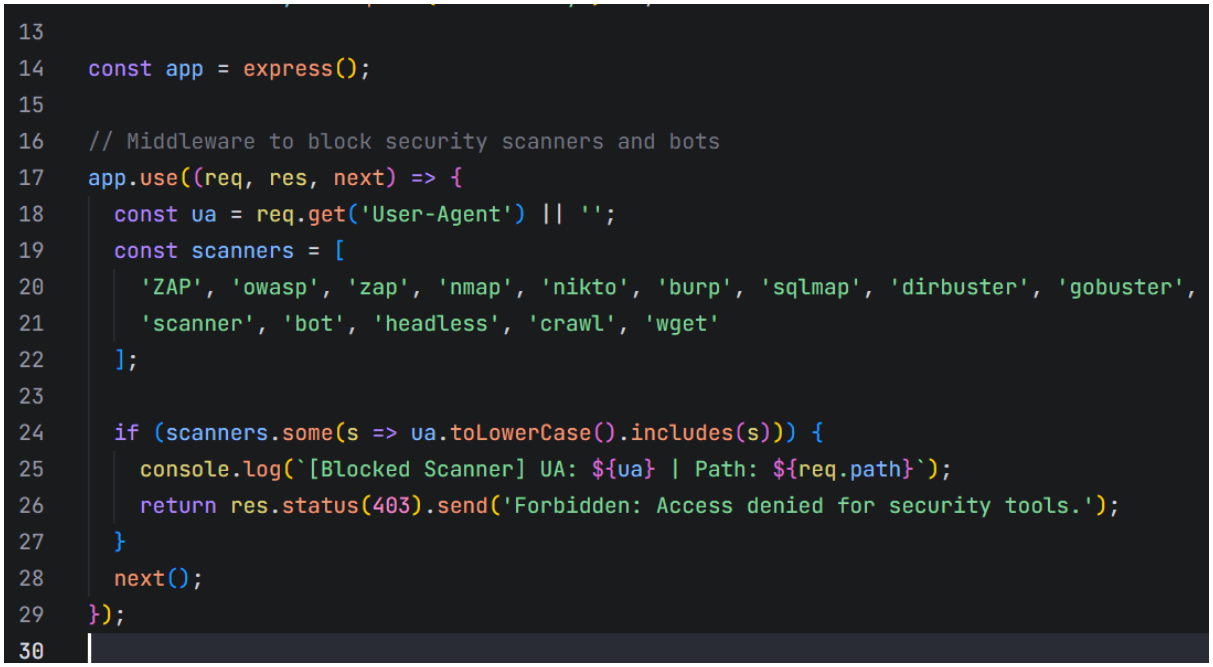

Standard Mode
Sites
Quick Start
Request
Response
Requester
application while proxying through ZAP.
The ZAP Heads Up Display (HUD) brings all of the essential ZAP functionality into your browser.
URL to explore:
https://ajmeats.dpdns.org
Select...
Enable HUD:
Explore your application:
Launch Browser
Firefox
You can also use browsers that you don't launch from ZAP, but will need to configure them to
https://m.stripe.com
https://r.stripe.com
https://m.stripe.network
https://mernapp-production-3fb
https://js.stripe.com
https://ajmeats.dpdns.org
http://https
History
Search
Alerts
Output
Filter: OFF
Export
ID Sou... Req. Timesta... Met... URL C... Reason ... Size Resp. ... Highest ... N... Tags
1 Pr... 2/13/26, 8:04:... GET http://https//ajmeats.dpdns.org 504 Gatew... 2... 185 bytes
3 Pr... 2/13/26, 8:05:... GET http://https//ajmeats.dpdns.org 502 Bad G... 1... 274 bytes
4 Pr... 2/13/26, 8:05:... GET https://ajmeats.dpdns.org/ 200 OK 6... 676 bytes Medium Script

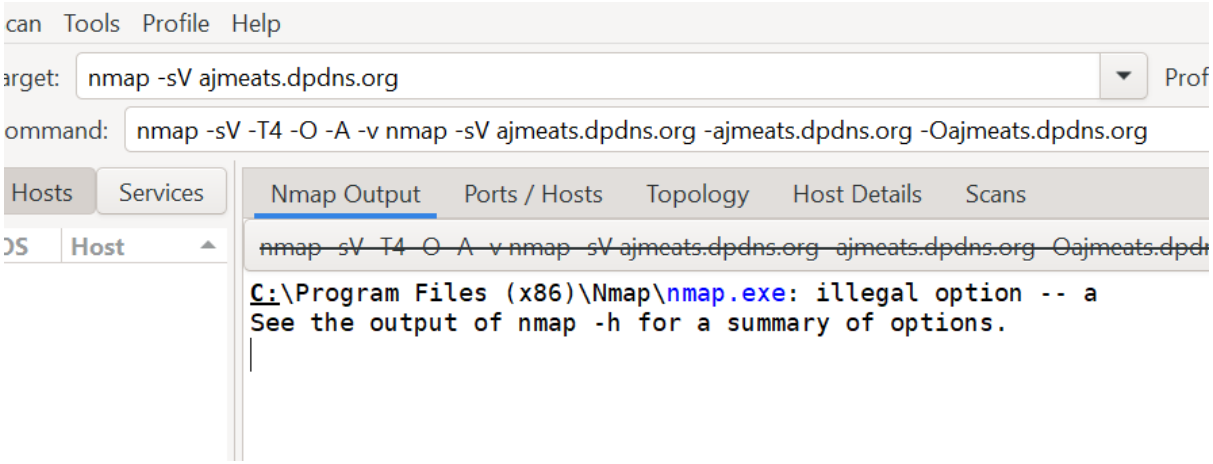
FIXED


JS index.js
ckend > JS index.js > ...
Changes were successfully applied. Please confirm.
61 }));
62 app.use(express.json());
63
64 app.get('/api/config/stripe-key', (req, res) => {
65 res.json({ publishableKey: process.env.STRIPE_PUBLISHABLE
66 });
67

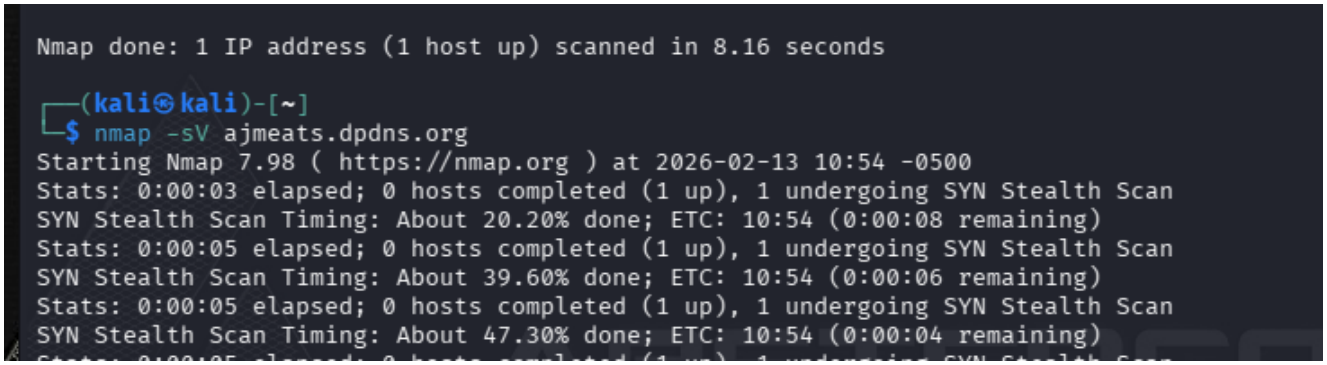

Alt+N Alt+Y
app.get('/api/config/stripe-key', (req, res) => {
res.json({ publishableKey: process.env.STRIPE_PUBLISHABLE_KEY || process.env.REACT_APP_STRIPE_KEY });
});
// Helper to resolve asset paths (checks current dir first, then parent)

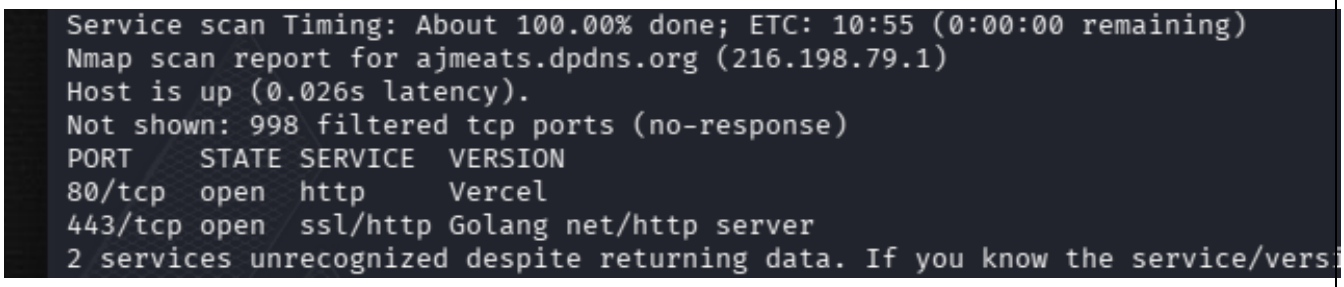

const stripePromise = loadStripe(process.env.REACT_APP_STRIPE_KEY || 'pk_test_placeholder');
7 let stripePromise = null;
8


13
14 const app = express();
15
16 // Middleware to block security scanners and bots
17 app.use((req, res, next) => {
18 const ua = req.get('User-Agent') || '';
19 const scanners = [
20 'ZAP', 'owasp', 'zap', 'nmap', 'nikto', 'burp', 'sqlmap', 'dirbuster', 'gobuster',
21 'scanner', 'bot', 'headless', 'crawl', 'wget'
22];
23
24 if (scanners.some(s => ua.toLowerCase().includes(s))) {
25 console.log(`[Blocked Scanner] UA: ${ua} | Path: ${req.path}`);
26 return res.status(403).send('Forbidden: Access denied for security tools.');
27 }
28 next();
29 });
30



APART FORM VERCEL THERE IS NO OTHER SERVICES MENTIONED





AUTOMATED ATTACK VIA ZAPROXY (METHOD NOT ALLOWED)

D	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
173	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/robots.txt?-d+allow_url_...	405	Method Not Allo...	147 ms	373 bytes	0 bytes
174	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/AJ.png?-d+allow_url_incl...	405	Method Not Allo...	166 ms	353 bytes	0 bytes
175	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/manifest.json?-d+allow_...	405	Method Not Allo...	148 ms	382 bytes	0 bytes
176	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static/js?-d+allow_url_in...	404	Not Found	111 ms	310 bytes	79 bytes
177	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static/css?-d+allow_url_i...	404	Not Found	125 ms	310 bytes	79 bytes
178	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static?-d+allow_url_inclu...	405	Method Not Allo...	148 ms	348 bytes	0 bytes
180	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static/js?-d+allow_url_in...	404	Not Found	46 ms	310 bytes	79 bytes
179	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static/css/main.0a489c6...	405	Method Not Allo...	145 ms	372 bytes	0 bytes
181	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/sitemap.xml?-d+allow_u...	405	Method Not Allo...	167 ms	348 bytes	0 bytes
183	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static/css?-d+allow_url_i...	404	Not Found	44 ms	310 bytes	79 bytes
182	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static/js/main.2d4a5525....	405	Method Not Allo...	166 ms	385 bytes	0 bytes
184	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static/js/main.2d4a5525....	405	Method Not Allo...	60 ms	385 bytes	0 bytes
185	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/sitemap.xml?-d+allow_u...	405	Method Not Allo...	80 ms	348 bytes	0 bytes
186	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/static?-d+allow_url_inclu...	405	Method Not Allo...	136 ms	348 bytes	0 bytes
187	2/13/26, 10:40:08 AM	2/13/26, 10:40:09 AM	POST	https://ajmeats.dpdns.org/static/css/main.0a489c6...	405	Method Not Allo...	157 ms	372 bytes	0 bytes
188	2/13/26, 10:40:08 AM	2/13/26, 10:40:09 AM	POST	https://ajmeats.dpdns.org/static/media?-d+allow_u...	404	Not Found	123 ms	310 bytes	79 bytes
161	2/13/26, 10:40:06 AM	2/13/26, 10:40:06 AM	GET	https://ajmeats.dpdns.org/manifest.json?-s	200	OK	56 ms	516 bytes	463 bytes
162	2/13/26, 10:40:06 AM	2/13/26, 10:40:07 AM	GET	https://ajmeats.dpdns.org/robots.txt?-s	200	OK	65 ms	506 bytes	67 bytes
163	2/13/26, 10:40:07 AM	2/13/26, 10:40:07 AM	GET	https://ajmeats.dpdns.org/static/css/main.0a489c6...	200	OK	225 ms	510 bytes	103,375 byt
164	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/?-d+allow_url_include%3...	405	Method Not Allo...	69 ms	349 bytes	0 bytes
165	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/?-d+allow_url_include%3...	405	Method Not Allo...	68 ms	349 bytes	0 bytes
166	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/about?-d+allow_url_inclu...	405	Method Not Allo...	64 ms	348 bytes	0 bytes
167	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/AJ.png?-d+allow_url_incl...	405	Method Not Allo...	150 ms	353 bytes	0 bytes
168	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/robots.txt?-d+allow_url_...	405	Method Not Allo...	153 ms	373 bytes	0 bytes
169	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/manifest.json?-d+allow_...	405	Method Not Allo...	160 ms	382 bytes	0 bytes
170	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/?-d+allow_url_include%3...	405	Method Not Allo...	147 ms	349 bytes	0 bytes
171	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/?-d+allow_url_include%3...	405	Method Not Allo...	154 ms	349 bytes	0 bytes
172	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/about?-d+allow_url_inclu...	405	Method Not Allo...	158 ms	348 bytes	0 bytes
173	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/robots.txt?-d+allow_url_...	405	Method Not Allo...	147 ms	373 bytes	0 bytes
174	2/13/26, 10:40:08 AM	2/13/26, 10:40:08 AM	POST	https://ajmeats.dpdns.org/AJ.png?-d+allow_url_incl...	405	Method Not Allo...	166 ms	353 bytes	0 bytes

#	Vulnerability / Finding	Description	Likelihood	Impact	CVSS (Approx.)	Risk Level	Status
1	Exposed Backend URL (Railway)	Frontend JS bundle exposed backend API endpoint (mernapp-production-3fbc.up.railway.app)	High	High	9.1	Critical	Mitigation Recommended
2	Potential CSRF on /api/user/location	POST request accepted without visible CSRF token	Medium	Medium	6.5	Medium	Hardened / Under Review
3	Directory Enumeration Results	No sensitive directories found via Gobuster	Low	Low	2.0	Low	Secure
4	Port Exposure	Only Port 443 open (HTTPS)	Very Low	Low	1.0	Informational	Secure
5	Database Exposure	No MongoDB endpoints publicly exposed	Very Low	High (if exposed)	0	Secure	Secure
6	SQL / NoSQL Injection	No injection vulnerabilities detected	Low	High	0	Secure	Secure
7	JWT Authentication	JWT functioning properly; no token tampering detected	Low	High	0	Secure	Secure
8	Automated Scan (ZAP)	No major vulnerabilities discovered	Low	Medium	0	Secure	Secure

Conclusion

This project involved performing a structured security assessment of the MERN stack application hosted on Vercel and Railway infrastructure. Multiple testing methodologies were applied, including reconnaissance, directory enumeration, JavaScript bundle analysis, API testing, and automated vulnerability scanning.

The assessment confirmed that:

- No critical network-level vulnerabilities were detected.
- No database endpoints were publicly exposed.
- No SQL Injection or NoSQL Injection vulnerabilities were identified.
- Authentication mechanisms using JWT are functioning correctly.
- Backend services are properly separated from frontend infrastructure.

Although minor hardening improvements are recommended, the overall security posture of the application is stable and follows modern web security practices.